IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Digital Object Identifier

Two Reachability-Gated Relations in MST-wi: Temporal Mismatch, Oracle Admissibility, and Measurement Limits

QIULONG LIN[1,2] AND XIAOYU KE[1] AND NANLEI CHEN[1,2]
[1]Yango University, Fuzhou 350015, China (e-mail: ubfbzu3@163.com; 5197211@qq.com)
[2]Sogang University, Seoul 04107, Republic of Korea

Corresponding author: Nanlei Chen (e-mail: ubfbzu3@163.com).

ABSTRACT Two system-agnostic relations in MST-wi, a Web-system metamorphic testing framework, gate their verdict on a *GUI-reachability model*—a crawl-time snapshot of what the interface permits—so what they establish about object-level authorization depends on when the model was read and what the adjudicator may consult. Source inspection shows the engine records only that a relation failed: promotion to a *proven violation* is added policy, stated here explicitly, so every false-proof count is reported together with the promotion policy that produces it. On a 576-record constructed study, a legitimate access whose grant lies outside the framework's record is reported as proven in 8 of 8 alarms, and the same failure appears when the grant is made between crawl and request. Once a readable, sound and independent oracle is supplied, the relation adds no adjudication information over the direct consumer of the same oracle and outcome—an identity on the constructed grid rather than a measured gain, and conditional on an oracle that none of the systems tested here supplies, where the direct comparator in fact produces more false proofs on the two products that expose a vulnerable path and ties it at zero on the third; and the published specificity of 99.81% is an input-level figure whose denominator cannot, on its own, expose a class-uniform failure. A temporal experiment on Gitea 1.22.6 and GitLab CE 17.11.7 commits 16 authorisation transitions over 712 cross-credential cells: 29 stale false proofs appear, and a recrawl with the same credentials removes 29. A real vulnerable/fixed pair (Gitea 1.20.5 to 1.20.6) shows the independent entitlement/effect gate reporting 1 true positive on the vulnerable build, 0 false proofs, and no alarm on the fixed build—a check of the gate, not a measure of detection effectiveness; across four seeded defect classes (9 instances) the studied relation fires only on the read class, a boundary rather than a tuning gap.

INDEX TERMS BOLA/BFLA, false proof, metamorphic testing, object-level authorization, oracle problem, specificity measurement

I. INTRODUCTION

BROKEN object-level authorization (BOLA), first in the OWASP API Security Top 10 [1], is the failure to enforce, per object, that the authenticated subject is entitled to the object touched. Detecting it is an *oracle* problem: vulnerable and correct systems answer with the same success status. Metamorphic testing is an established approach to that oracle problem, and MST-wi [2] is a substantial system-agnostic instance for Web systems: 76 system-agnostic relations (MRs) over 55 Web-specific predicates, reported to detect 85% of vulnerabilities with specificity 99.81%.

This paper asks what happens to that claim when the asserted property is object-level authorization. The predicates do not express entitlement; they express what their authors call reachability—what "the functionality exposed by the user interface" permits—with the role-to-resource mapping "**not always available**" [2]. Delegating adjudication to a reachability model makes the moment the model was read part of the specification, so we instantiate the predicates *as defined*—over a model recorded at crawl time—and drive the real systems through three phases: crawl, change authorization out of band, execute. The middle phase is the one the original evaluation does not exercise. The constructed study isolates the same defect without time: it seeds a share outside the framework's record and asks whether a gate that cannot see it still calls the access legal. We ran a 576-record scenario/configuration study on a constructed system and a controlled study on Gitea 1.22.6, extended, with a methodological variation, to Gogs 0.14.3 and GitLab 17.11.7. We propose no mechanism; three findings survive, one per

research question (Section VI).

Research questions and findings

(1) *Temporal (RQ1).* Does the crawl-time reachability model remain sound for authorization decisions after an out-of-band authorization change? No: the adjudication as designed produces *deterministic false proofs*—a legitimate access the model cannot see is reported as a proven violation once an alarm is promoted, at rate 1.00 in the constructed study and 0.40 on Gitea. The predicate's two representations cannot describe either grant, so the error is a property of the *reading*, not of the system. The alarm and the count of proven violations are separable: no policy changes what the relations return, while the number of contradictable claims runs from none, through 29 under promote-every-alarm, to 13 under the oracle-gated variant (Table 16). (2) *Oracle (RQ2).* When a readable, sound authorization oracle fully resolves the proposition under test, does the metamorphic step add incremental adjudication information beyond the oracle and the request outcome? Not on the evaluated domain—and there the answer is a construction rather than a measurement: the direct adjudicator is built from the same entitlement and outcome quantities that define the reference label, so the two rules *cannot* part where the oracle agrees with the ledger (Section VI-B). Whether that domain is reached is empirical: its antecedent is an oracle that is readable, sound *and* admissible, and no system studied here supplies one (Section VI-E). Where it fails, substituting the direct adjudicator makes matters worse—false proofs 17 against 8 on Gogs and 18 against 5 on GitLab (Table 12). (3) *Measurement (RQ3).* Can a pooled specificity over follow-up inputs provide stable scenario-level resolution for a failure confined to a class? No: unaffected inputs dilute it, and a class the crawl never reaches contributes no negative inputs, so the figure is silent about that class (Proposition 1). The published labels are manually verified; the limit is the denominator and the input set.

What is new here, and what is not

Two of the ingredients are established and we neither re-derive nor claim them: that a crawl-time reachability model can go stale or under-enumerate (the predecessor attributes its residual false alarms to crawler coverage [3]), and that a reachability predicate is not an entitlement predicate [2]. A third—that a pooled specificity is a weak instrument for class-level failure—is familiar from the measurement literature [4]. Section IX sets each claim beside the prior position it extends and names the check that would count against it. What is new is the following. (i) *A representation argument, not a coverage observation.* The false proof is a deterministic consequence of which of the predicate's two representations a grant fits, and the variable that governs it is *when* the model was read rather than what was visible—which also tells staleness (repairable by recrawling) from channel blindness (not repairable by the same recrawl; Section V). (ii) *The alarm/policy separation, quantified.* The technique returns an alarm; treating it as a violation is a policy we state, and we report what four policies make of the same alarms (Section VI-A). (iii) *A conditional equivalence with a named antecedent.* Promotion through the relation is a subset of direct adjudication once an admissible oracle exists, which bounds the technique's added value from above; we apply an explicit admission test that, on the systems tested, refuses it (Section VI-E). (iv) *A denominator argument about the published metric.* A specificity pooled over the inputs a crawl could enumerate cannot register a failure confined to a class that crawl did not reach (Proposition 1). Sections III–VII carry the evidence, which supports these four items rather than standing as further headline claims.

II. BACKGROUND: THE PREDICATES IN QUESTION

For authorization there is no intrinsic oracle: vulnerable and correct executions both answer with success. Metamorphic testing replaces the absent oracle with a relation over multiple executions—and the relation is only as sound as the assumptions it needs. MST-wi [2], building on Mai et al. [3], crawls a per-user model of reachable URLs and contents, then checks MRs against that recorded model. In the artifact we inspected, the predicates guarding an authorization MR read either the recorded model directly (`cannotReachThroughGUI`, `userCanRetrieveContent`) or static subject properties; we call a predicate *GUI-derived* when its value is a function of the crawl.

Why the false proof is a design consequence, not a bug

Suppose a subject can reach an object at crawl time, so the record stores it as reachable. Afterwards an administrator grants the same subject access to a *further* object through a channel—an administrative endpoint or a share link—that a crawl of the subject's own interface never traverses. At execution the guard still reads the frozen record, finds the new object absent, and concludes the subject cannot reach it; the relation then treats the successful response as a violation. Every step is exactly as specified, so no guard is miswritten: the predicate's two representations—''reachable as recorded'' and ''not reachable as recorded''—cannot express ''reachable, but only after the record was taken''. The two cures that follow are different and must not be conflated. When the grant makes the object listable through the subject's own interface, a *re-enumeration with the same credential* picks it up: in the temporal experiment such a same-credential recrawl repairs all 29 stale cells, with no elevation of credential involved. When instead the object never enters that crawl source at all, no amount of re-enumerating repairs the model, and that is the boundary the Gogs residual cells in Section V-B illustrate. The reported specificity does not register the failure either, because it is a property of the adjudication's *reading moment* rather than of any single generated input. This is the mechanism the rest of the paper measures, and the reason we speak of a boundary of the specification rather than a defect of an implementation.

Two census figures frame what follows. Over the 76 catalog MRs, 38 (50.0%) gate on a GUI-derived or identity predicate, and only 14 (18.42%) are authorization-relevant

by name or action—an upper bound; a full-text search for predicates reading the *state of an authorization artifact* (ownership, permission record, share) finds 0 occurrences. Within the examined vocabulary, the authorization MRs have no authorization-artifact state observable to decide on, and the reading moment enters the specification: if an entitlement changes after the crawl, the MR evaluates reachability against a world that no longer exists. Our experiments separate the two ways the world can move under the model: the constructed study varies what the gate can see, and the temporal study varies when the model was read.

III. PROTOCOL: A FAITHFUL THREE-PHASE INSTANTIATION

Two instruments carry distinct claims. The three-phase instantiation below carries the quantitative results of Sections IV and V; a direct execution of the library's own relations—source unmodified, with the build configuration and a replay harness added—carries one qualitative claim, that the published code produces the false proof we predict (Section VII-C). Neither instrument is a benchmark or yields a claim of detector accuracy; the standing scope and non-claims are collected in Section VIII.

Three phases

(A) *Crawl*: record, per subject and object, whether the subject can reach the object through the interface, and freeze that record—the model the predicates will consult. (B) *Change authorization out of band*: alter a (subject, object) entitlement through a channel outside the crawled interface; the frozen model is not updated, because the architecture has no mechanism to update it. (C) *Execute the MR* against the current state while the predicates consult the frozen model. Phase B is what the original evaluation does not exercise, and where the reachability stand-in can be wrong (Fig. 1).

The protocol is frozen, and it was amended twice

We describe the protocol as *frozen* rather than as preregistered, because only the first word is true here. The base protocol was pinned by digest (hash `d09d904707aa`) before the first cell of the first run was observed, and the runs carry that digest in their records; but the rules a run followed are not all rules that were written before any observation. Amendment A1 demoted the secondary calibration vantage from a pass/fail gate to a recorded diagnostic, and scopes to the second product only. Amendment A2 records a nonlocal side effect found in an archived pilot—removing a user's group membership also removes that user's direct membership in the group's projects—and it is why the ledger is *not* patched with a rule learned from observation: encoding it would make E depend on a behaviour learned from Y. The pilot is archived and no cell of it enters any denominator. Which rules a run followed is recoverable from its `amendment_id` and `amendment_sha256` fields rather than from a timestamp, and A2's self-reported write time falls after the reported run's last probe; we retain the field as recorded rather than correct it in place, because doing so would change the digest the run binds. What no amendment touched is fixed as written: the ground-truth source, the abstention exits, the *unknown* policy, the denominator conventions, and the prohibition on letting Y or O reach E. The two rules left genuinely open—how an *unknown* entitlement is scored, and how an alarm becomes a claim—are treated as explicit choices with their sensitivity reported (Tables 1 and 16).

Alarm, claim, and truth

Three objects are kept apart throughout. The *alarm* A is what the technique returns: $A = y \wedge \neg m$, the conjunction of a successful request with the absence of the subject from the frozen reachability model. It asserts that a metamorphic relation failed; it asserts nothing about authorization, and it has no ledger, no expected decision and no verdict behind it. A *claim* H is the promotion of an alarm to "violation-proven" by a policy P that we state rather than inherit—the catalogue's purpose invites that reading, but its code does not perform it. The *truth* T is computed independently, from the ledger's entitlement E and the content-level effect Y. Promotion is therefore an adjudication policy layered on the technique's native output, not part of it, and the reported false-proof count is a property of the pair (A, P). Section VI-A quantifies how far the choice of P moves the published conclusion, and Section VII-C applies the same separation to the unmodified library.

Adjudicator variants and internal controls

A raw alarm becomes a verdict through an exception policy, and different policies are different methods: `raw` (no exception handling—the baseline any filter must be read against); **v1** (an *existence* filter: any sharing record on the object downgrades the alarm); **v2** (existence plus operation coverage); **v3** (state-calibrated—consults a readable authorization introspection surface, abstains when none is readable); `degrade_all` (the floor: abstain on every alarm); `direct_only` (authorization surface only, access outcome ignored—a deliberately unfair control); and `direct_observe` (a succeeding plain request checked against the same readable surface), which decides the negative result.

Ground truth

Every quantitative result names its ground-truth source, and it is never the detector under test. E (entitlement) is read from the runner's own ledger of authorization events, recorded at mutation time and independent of the interfaces under test; Y (effect) is the content-level outcome of a probe by the subject's own credential (for writes, owner-side re-reads rather than status codes); O (the introspection surface) is diagnostic only and never substitutes for either. That ledger and content probes agreed cell by cell is a measured result, and the oracle's disagreements with the ledger are reported as findings, not absorbed.

Unknown and failure handling

The failure modes are distinct and stated as implemented. *Transport-level retries:* a request that fails to connect

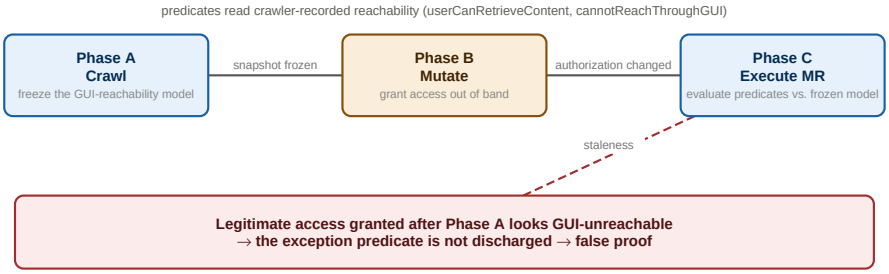


FIGURE 1. **The three-phase instantiation. The predicates consult the reachability model frozen in Phase A. A Phase B entitlement change made outside the crawled interface leaves that model stale, so in Phase C a legitimately granted access has no representation but "cannot reach"; the exception predicate is not discharged and the alarm is promoted to a proven violation—the false proof of Finding 1.**

(status 0) is re-issued up to two more times. *E undefined:* an entitlement-changing transaction that is refused or does not commit is issued once, marked *failed* (`mutation_failed`) and excluded from every denominator, with the exclusion count reported. *Y undefined:* an execution error or timeout leaves the cell unscored, never coerced to *allow* or *deny*—in the native engine run, 11 relations were attempted, 10 executed and 1 errored. *O undefined:* the state-calibrated method abstains on those cells by construction, and the channel table reports them in an explicit *unanswered* column (56 Gitea and 72 GitLab cells for the anonymous reader, 0 on Gogs' authenticated reader) instead of absorbing them into *deny*.

The unknown *rule is a scoring choice, and both calibers are reported*

E is `unknown` when an object is absent from the ledger model. For a modelled object with no recorded granting channel the frozen text promised `unknown`, while the shipped implementation returns a genuine *deny*, reserving `unknown` for the absent-object case and for deploy keys. We disclose this divergence rather than reconcile it silently, and report both readings. Closed-world default deny is what the main analysis uses; the literal reading marks those cells E-unknown and excludes them from scoring. Neither reading touches a single alarm, because every stale false proof falls on an E-allow cell: the 29 stale false proofs and the 0 live ones are identical under both. What the rule moves is the population the metrics are computed over, far enough to change the reported rate by a factor of about four. Under default deny all 712 committed cells are scorable, the 29 false proofs are a share of 4.1% of that set, and specificity is 95.9%; under the literal rule the scorable set shrinks to the 171 cells whose entitlement the ledger fixes, the same false proofs are a share of 17.0% of it, and specificity falls to 83.0%. The literal rule is not an improvement: it raises the observed share only by discarding the 541 cells on which the adjudication happened to be right. Precision is zero in both calibers and recall is undefined in both, because this grid contains no violation. Table 1 gives the two confusion matrices side by side.

The closed-world assumption is bounded, not verified

Default deny is sound only if the ledger enumerates every granting channel, and that assumption cannot be established from inside the run. It holds *by construction* for the frozen topologies used here: every grant written in the setup phase went through a modelled channel (owner, organisation owner, site administrator, public visibility, direct collaborator, team or group inheritance). It is also partially self-checking: an unrecorded grant the system in fact enforces would surface as $E \neq Y$ on a probed cell, and $E = Y$ holds on all 712 of 712 cells—a measured result rather than a definition, since the oracle disagrees with E on 53 cells in the same runs. What the check cannot see is an unrecorded grant on a cell nobody probed, a channel that exists but is never exercised, or a grant the system also fails to enforce; deployment adds directory and SSO group synchronisation, deploy keys, external identity providers, and grants written straight to the database.

Table 2 answers a narrower question by intervention. For each channel in turn its grant rows are deleted from every cell's recorded basis list and the damaged entitlement is scored, with the scoring executed rather than re-derived: an alarmed cell is passed to the same classifier the runs use, and an unalarmed cell counts as an *apparent* false negative when the damaged entitlement is deny and the observed effect is allow. The closed-form counts are asserted equal to the executed ones. Nothing a subject is permitted to do changes, because a ledger is a record and not the system: every cell that moves here remains a legitimate access, and the 29 false proofs are not repaired by its moving, only re-described. The alarm cannot move either—it is computed from the crawl snapshot, the probe response and the role, and never reads the ledger. The intervention adds one premise: deleting one channel's rows does not cascade into another channel's entitlement, and assertions on the 5 multi-channel cells verify only that the implementation preserves an alternative recorded basis. Omitting public visibility re-labels 97 legitimate cells as deny; against that label 11 existing alarms score as *apparent* true positives and 86 legitimate accesses as *apparent* false negatives, and the reported false-proof count falls to 18. Site administration is the extreme case, where 14 of the 29 alarms become apparent true positives and no access

TABLE 1. The 712 committed cells scored under the two *unknown* rules, with all four entries of the confusion matrix given (TP, FP, TN, FN). A positive is a violation (ledger deny, effect allow) and a negative is every other cell; in these runs $E = Y$ on every cell, so the design contains no positive and recall has no denominator (marked N/A, not 0). Specificity and precision are therefore properties of the negative set each rule admits, and that set is what changes between the two calibers: the claims do not move, the denominator does.

		cell population			confusion matrix				rates (%)	
caliber	product	scorable	negatives	E unknown	TP	FP	TN	FN	spec.	prec.
default deny	Gitea	280	280	0	0	13	267	0	95.4	0.0
default deny	GitLab	432	432	0	0	16	416	0	96.3	0.0
pooled		712	712	0	0	29	683	0	95.9	0.0
literal	Gitea	77	77	203	0	13	64	0	83.1	0.0
literal	GitLab	94	94	338	0	16	78	0	83.0	0.0
pooled		171	171	541	0	29	142	0	83.0	0.0

recall: N/A in both calibers (0 positives in the design; the temporal grid measures the false-proof face only)

becomes an apparent miss. Cells resting on a second channel are untouched: 2 on public visibility, 3 on direct collaborator and 3 on team inheritance.

With the sample, the alarms, the probe responses and the promotion decisions all held fixed, and only legitimate grant rows deleted under default deny, a damaged label can lower the reported false-proof count but cannot raise it: incompleteness hides false proofs from the measurement rather than exposing them. A promotion policy that reads the ledger rather than being fixed in advance acts on those contradictions, so the 11 re-labelled cells are exactly the alarms a ledger-gated policy would promote, and its zero in Table 16 is a statement about a sound ledger rather than a property that survives a damaged one.

Scenarios and design cells

Seven scenarios separate the reasons an MR can misfire: no shared object (S1, S2); an in-band legitimate share visible in the framework log (S3); an unshared plain violation (S4); a read-only share plus a write violation (S5); an out-of-band share invisible to the framework log (S6); an out-of-band share plus a genuine violation (S7). They are crossed with 8 configurations defined by instantiation mode, enforcement correctness, channel visibility and introspection availability. The nominal cross is 7×8 combinations, but four are inadmissible: the two no-exception configurations (`V-noE3-vuln0`, `V-noE3-vuln1`) are undefined under S6 and S7, where the legitimate-sharing exception does not apply. This leaves 52 admissible design cells.

Table 3 gives the complete cross factor by factor; the cell-by-cell census ships with the artifact. Two dimensions are nested rather than crossed: channel visibility exists only where the application offers a sharing facility, so the two no-exception configurations sit outside it; and the listed level was instantiated only with introspection on, so *listed and unreadable* is untested rather than shown to be impossible.

A. WHAT THE DESIGN VARIES, AND WHAT IT CAN IDENTIFY

Because the exception policy and the reachability reading are thrown per record rather than held per configuration, every record has a partner that differs in that switch alone. Table 4 pairs records on configuration, scenario, relation and the other switch, and reports the discordant pairs.

Three results come out of it. First, the reachability reading moves the outcome in one direction only: of 288 blind–aware pairs, 12 lose a false proof when the reading is aware and 0 gain one, while the same switch costs 3 detections and gains none. It is a statement about what the gate can see, not about when authorization changed: the constructed study seeds the out-of-band share before the run and never mutates entitlement inside it, so the temporal reading belongs to Section V. Second, the exception ladder has *no* discordant pair at either step: raw and e1 differ on no individual record, and neither do e1 and e1e2, so the two exception predicates are, on this system, inert rather than merely weak. Third, oracle readability acts only where there is something to resolve: in the violating configuration 54 of 78 paired records change from `violation-proven` to `indeterminate` when the surface is switched off, none the other way; in the correct configuration 78 pairs change not at all.

What the design cannot identify follows from the two nested dimensions. The interaction between channel visibility and readability is not estimable, because the listed-and-unreadable cell was not run; the readability contrast is identified within unlisted only. The legitimate-sharing mechanism is not identified under S6–S7, where it is undefined by construction, so the out-of-band estimates are for the e3 configurations alone. The two relations are not orthogonal to scenario—S5 carries MR-002 only—so we never compare relations with each other. What remains is balanced: every (exception policy, reachability reading) slice holds 52 readable and 44 unreadable records.

Statistical treatment

The 576 records are not independent observations: each admissible cell is expanded deterministically by the factors actually varied—2 metamorphic relations (MR-002, MR-004), 3 exception-policy variants, and 2 reachability readings. Only MR-002 applies under scenario S5, so 8 S5 cells carry one relation while the remaining 44 carry two, giving

$$576 = 44 \cdot 2 \cdot 3 \cdot 2 + 8 \cdot 3 \cdot 2.$$

The records are a fixed enumeration, not a sample: we report counts, rates and design-cell denominators and attach no inferential interval to any of them. Where a headline count has a

TABLE 2. Scoring against a damaged reference label: what one missing granting channel does to the reported numbers. The intervention edits the ledger, not the system. The damaged reference labels are scored using the procedure described in Section III, with the cell sample, the alarms, the probe responses and the promotion decisions held fixed. Under the damaged label *re-labelled* counts the cells that now read deny; of those, *apparent TP* counts the alarmed ones (a false proof now scoring as a confirmed violation) and *apparent FN* the unalarmed ones (a legitimate access now scoring as a missed violation); *reported FP* is the false-proof count the damaged label still measures.

channel omitted	resting	re-labelled	apparent TP	apparent FN	reported FP
none (as run)	171	0	0	0	29
public visibility	99	97	11	86	18
direct collaborator	50	47	2	45	27
team / project group	11	8	2	6	27
site administrator	16	14	14	0	15

resting counts cells per channel and overlaps: 5 cells rest on two channels, so the column sums to $171 + 5$ against the 171 cells scored.

TABLE 3. The complete design. Every record is one point of the cross scenario × configuration × relation × exception policy × reachability reading; the last two are deliberately thrown switches, not replicates, so the enumeration carries no sampling error. "Records" gives the count at each level of the factor, and the last column states what the factor is crossed with. Only the listed-with-introspection cell is missing, and Section III-A says what that costs.

factor	levels	how it is varied	records	crossed with
scenario				
scenario	7	the origin of the share and the ground truth: no shared object (S1, S2); a share made through the framework's own facility (S3); an unshared violation (S4); a read-only share plus a write violation (S5); a share made out of band (S6); out of band plus a violation (S7)	48–96	all configuration levels where admissible
configuration				
instantiation	2	whether the application offers a legitimate-sharing facility at all (e3) or not (no-e3)	468 / 108	enforcement, visibility; undefined under S6–S7
enforcement	2	the application either enforces object ownership (correct) or serves the object to any authenticated subject (violating)	288 / 288	full
channel visibility	2	whether the share is written to the framework's own log (listed) or not (unlisted)	420 / 156	e3 only; no share exists to list under no-e3
introspection	2	whether the application exposes a readable per-object share list (on) or not (off)	420 / 156	*listed exists only with introspection on*
expansion				
relation	2	the metamorphic relation evaluated: MR-002 or MR-004	312 / 264	S5 carries MR-002 only
exception policy	3	how many MST-wi exception predicates are thrown before the relation fires: none (raw), one (e1), two (e1e2)	192 / 192 / 192	full
reachability reading	2	whether the gate consults a constant unreachable list (blind) or the listing current when the relation runs (aware). The two differ in what the model can see, not in when authorization changed	288 / 288	full

$44 \cdot 2 \cdot 3 \cdot 2 + 8 \cdot 3 \cdot 2 = 576$ records over 52 cells

TABLE 4. Attribution by matched pairs. Each row compares two records that differ in exactly one switch, holding configuration, scenario, relation and exception policy (or reachability reading) fixed, so the contrast cannot be carried by any other factor. "First only" and "second only" are the discordant pairs: a one-way count means the switch moves the outcome in one direction only, and a zero means it does not move it at all. The two switches that were hypothesised to matter are separated in the first two blocks; the third shows the effect is confined to violations that a readable surface can resolve.

contrast	outcome	pairs	first only	second only	both	neither	reading
blind → aware	false proof	288	12	0	24	252	the aware reading removes 12 false proofs, adds 0
blind → aware	violation detected	288	3	0	105	180	and costs 3 detections
raw → e1	false proof	192	0	0	20	172	no discordant pair: the ladder changes nothing
e1 → e1e2	false proof	192	0	0	20	172	no discordant pair: the second predicate adds nothing
unreadable → readable	verdict of violation-proven	78	0	54	0	24	all 54 changes are detections gained, none lost
unreadable → readable	verdict of violation-proven	78	0	0	0	78	no change where there is no violation to detect

small denominator we lead with the larger-denominator count of the same phenomenon (the anchored rate, 2/2, is unstable by construction). The replication ceiling is one constructed system, 52 cells, 2 MRs, three real systems.

IV. SYNTHETIC STUDY: WHERE THE ADJUDICATION GOES

This study supplies the evidence for Findings 1 and 3, and the ablation behind Finding 2; the findings themselves are stated in Section VI.

A. DEFINITIONS

Confusions are defined over the *verdict*: a *false proof* is the strict subset of false alarms promoted to `violation-proven`—an error the tool asserts as established. We also report the *detection-keep rate* (share of fired true violations still alarmed) and count only abstentions justified by an unreadable oracle.

B. WHAT THE CONSTRUCTED SYSTEM SHOWS

Attribution is clean

On the anchored configuration the false-proof rate is 2/2. With that denominator the rate is unstable by construction, so we attach to it neither an interval nor a percentage. With the legitimate-sharing exception absent the rate is 0.00 (0/2), and with the channel listed it is 0.00 (0/2). The phenomenon is therefore attributable to a legitimate sharing exception the MR cannot know about, not to execution itself. A 2×2 grid over channel visibility and enforcement correctness tells the same story: on unlisted cells the false-proof rate is 44.44% with no false negatives; on listed cells it drops to 0.0% but one of the five violating records in that cell is a false negative (4 detected, 1 missed). Avoiding false proofs and keeping detection are controlled by one switch, and the switch cannot be thrown both ways at once.

Filtering exceptions does not help

The exception ladder—raw, one exception level, two—leaves the anchored rate at 1.00, 1.00, 1.00: it does not move. The exception levels we can express are themselves functions of the same reachability model, so they inherit its blind spot.

The framework cannot see the channel that matters

When a legitimate share was created through the framework's facility, its log holds 12 records and v2 produces 0 false proofs (rate 0.00); when the same share was created out of band, the log holds 0 records—no evidence the share exists—and v2's false proofs rise to 8 of 8 alarms (rate 1.00), every one promoted to `violation-proven`. The MR cannot distinguish ''this share does not exist'' from ''this share exists but was not crawled''; its only representation of the first is the second. On S7, where a genuine violation *does* sit behind the out-of-band share, the same blindness makes the alarm right for the wrong reason (6 of 6 reached)—the coincidence an input-level metric cannot unpack. The same tension appears within a single MR: for S3 (must not alarm) vs S5 (must alarm), no configuration in the family is correct on both—the choice of switch is a choice of *which* error to make (Fig. 2).

Stratification, not aggregation

Aggregated, v3 looks best (recovery 1.00 against `degrade_all`'s keep rate 0.00); the reading is wrong, because v3 changes regime with oracle readability. Stratified over the 36-record readable grid and its 18-record absent grid (a representative grid; Section IV-F re-runs the comparison over all 576 records): on the readable grid it removes v2's false proofs (0 vs. 4) while still alarming on all 9 fired true violations; on the absent grid (18) it alarms on none of the 5 true violations v2 still alarms on, abstaining 13 times. Pooling converts honest abstention into apparent performance.

C. FAIR ABLATION: DOES THE METAMORPHIC STEP ADD ADJUDICATION INFORMATION?

Our earlier iteration proposed v3 as a mechanism, resting on the claim that the metamorphic step was necessary. We tested that claim by removing the step and substituting the most naive consumer of the same observable. The distinction matters: `direct_only`—oracle yes, access outcome ignored—produces 108 invalid violation claims (binary false positives, none of them strict false proofs; every one falls on a request the server denied) against v3's 0 (precision 0.50), and rejecting it proves nothing because it discards half the evidence. The fair control is `direct_observe`: a succeeding ordinary request plus the same readable surface, consuming exactly what the MR-based adjudicator consumes.

Table 5 concludes the ablation. The first three rows are a ladder: `raw` yields 120 false proofs; the existence filter v1 cuts them to 60 but pays by abstaining on true violations (FN rises to 18); adding operation coverage (v2) restores the true violations (FN 3) at the same false-proof count. v3's zero in the FP column is bought with the FN column: its precision of 1.00 costs 111 false negatives (F1 0.65). `direct_observe` has 0 false proofs against v3's 0—a tie at zero, not a win. The comparison is paired, so we report the discordance: over the 216 true-violation records the two methods disagree on 3, and on every one `direct_observe` is right—its reachability precondition blocks a detection the plain request makes. No margin was pre-specified, so no hypothesis-test claim attaches. On these oracle-available scenarios we find no incremental adjudication advantage of the metamorphic step over direct observation.

D. COVERAGE, SELECTIVE RISK, AND WHAT A POOLED PRECISION DOES NOT SAY

Table 5 scores the alarm. A method that abstains is counted as not having alarmed, so the abstention lands in TN or in FN: right for ''does the tool report a violation'', wrong for ''does the method decide''. Table 6 rescores the same 576 records on the decision. v3 decides 348 of them—coverage 60.42%—abstaining on 228 (108 on true violations, 120 on records with none), and its precision of 1.00 and selective risk of 0.86% are properties of that coverage. The abstentions are not all of one kind: 156 are structural, and the other 72 fall on a *readable* surface that shows a covering grant—a non-decision, not a correct rejection. Of the 60 false proofs v2 promotes, v3 declines 24 because the surface is unreadable and 36 because it shows the grant; only the first are the availability gate the method is named for. With the 3 records on which no alarm is raised at all, v3 leaves 111 of 216 violations unproven.

`direct_only` decides 312 records and is right on 108 true violations; its 108 invalid claims are not strict false proofs, since every one falls on a request the server denied (108 without violation effect, 0 on an allowed request), and its selective risk is 34.62%. `direct_observe` decides 312 records—coverage 54.17% —abstaining on 264 (108 on true violations), detects 108 and raises no false proof; on the 240 records both decide, it detects 3 more violations than v3, and the remaining difference in pooled recall is coverage, not accuracy. Selective risk is not a ranking: `degrade_all` has a selective risk of 1.23% at 42.19% coverage and detects no violation. We therefore report precision and selective risk with coverage attached and never compare them at unequal

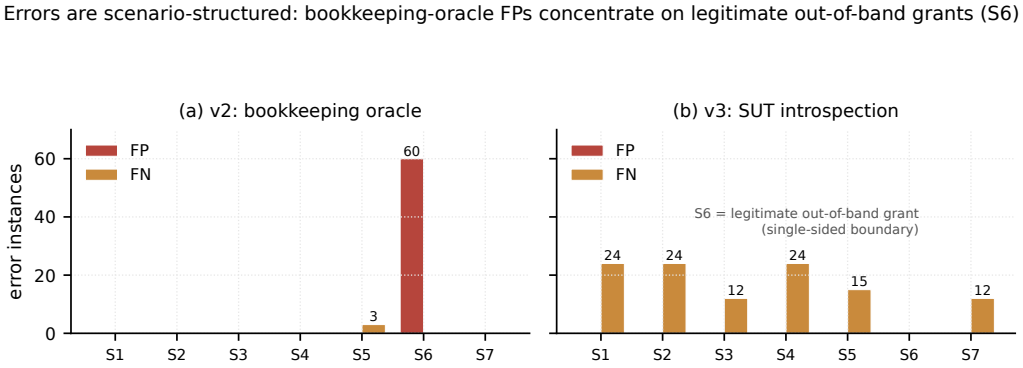


FIGURE 2. **The errors are scenario-structured. The bookkeeping oracle's false proofs concentrate on S6, the legitimate share created out of band and so invisible to the framework's own log, while the state-calibrated v3 abstains in two places: where its surface is not readable, and where a readable surface shows a covering grant it declines to override. Both panels are recomputed from the same records as Table 5.**

TABLE 5. **End-to-end confusion matrix over all 576 records (216 true violations, 360 correct cases). The records are a fixed enumeration, not a sample, so no inferential interval is attached.** `direct_only` **is a deliberately unfair control, shown to make that point.**

method	TP	FP	FN	TN	precision	recall	F1
raw (no exception handling)	213	120	3	240	0.64	0.99	0.78
v1 (existence filter)	198	60	18	300	0.77	0.92	0.84
v2 (existence + coverage)	213	60	3	300	0.78	0.99	0.87
v3 (state-calibrated)	105	0	111	360	1.00	0.49	0.65
`degrade_all`	0	0	216	360	—	0.00	—
`direct_only`	108	108	108	252	0.50	0.50	0.50
`direct_observe`	108	0	108	360	1.00	0.50	0.67

coverage.

Cost

The enumeration issues 4833 requests over 576 records (median 8 per record, range 7–10); the oracle-consuming methods add 104 probe GETs over 52 cell-operations, one request per record and 0.18 amortised. The temporal loop of Section V runs a median of 21.2 s per transition on Gitea and 7.3 s on GitLab, of which the mutation is 0.10 s and 0.09 s, the recrawl 0.38 s and 1.35 s, and the cross-credential probe sweep 20.7 s and 6.0 s. Re-reading every credential is what costs.

E. A PUBLISHED METRIC'S RESOLUTION: WHAT A POOLED SPECIFICITY CANNOT REGISTER

The published figure is a specificity, not a false-alarm share of all inputs. MST-wi defines it as $\mathrm{TN}/(\mathrm{TN}+\mathrm{FP})$ over the follow-up inputs F, and states that each reported MT failure was manually verified [2]; the labels are therefore not the crawler's own answer. With $\mathrm{FN} = 0$ throughout that evaluation, the figure equals $1-\mathrm{FP}/(F-\mathrm{TP})$: the denominator is the *negative* follow-up inputs, not every generated input.

Such a figure cannot register a failure confined to a class, for two reasons that are arithmetic and coverage rather than labelling. First, the figure is pooled: a class contributing a share w of the negative inputs can move it by at most w, so a real class-uniform failure is diluted by unaffected inputs. Second, the follow-up inputs are those the crawl could enumerate; a class of legitimate access the crawl never reaches contributes no negative inputs and cannot enter the ratio at all. The second point is the measurement face of Finding 1, and it is a statement about the coverage of the input set, not about the correctness of the human labels. We cannot reproduce the engine's exact input count, so we substitute no value for the published figure.

A bounded formal remark on the denominator

Write $S = 1 - FP/N^-$, where $N^- = \mathrm{TN} + \mathrm{FP}$ is the number of negative follow-up inputs the run generated, and define $S_{\mathrm{scenario}} = 1 - N_{\mathrm{failed}}/N_{\mathrm{scenarios}}$, where N_{failed} is the number of scenarios in which the failure is uniform. S_{scenario} is an analytical construct for characterising resolution, not a proposed replacement metric; the remark concerns resolution, not estimator consistency or power.

Proposition 1. Fix $f \geq 1$ failed scenarios, each producing a fixed number $a \geq 1$ of false-alarm inputs, and $s \geq 1$ successful scenarios each contributing m additional alarm-free negative inputs. Then $S = 1 - (fa)/(fa + sm)$, which tends to 1 as $m \to \infty$, while $S_{\mathrm{scenario}} = 1 - f/(f+s)$ is independent of m and strictly less than 1.

Proof. $N^- = fa + sm$ and $FP = fa$; substitution gives the first expression, which increases monotonically in m to 1, whereas the scenario-level figure depends only on the fixed counts f and s. □

Corollary 1. No fixed input-level specificity threshold $t < 1$ can guarantee that a scenario-uniform failure will push the metric below it: choosing $m > (fa/s) \cdot t/(1-t)$ drives S above t while the f failing scenarios remain failing.

The finding is therefore about what the published figure can be asked to show: it is pooled over classes, and computed

TABLE 6. The same 576 records scored on the decision rather than on the alarm. Coverage is the share of records on which a method asserts either a violation or its absence; the abstention column gives the count and, in parentheses, its rate, split by whether the record held a true violation. *FP claims* (invalid violation claims) and *misses* are the two error counts among decided records, and *risk* is their sum over that denominator—comparable across methods only at equal coverage, which these methods do not have. A passed record is one on which the method did not fire, not a certificate of safety. `direct_only` and `direct_observe` abstain when the probe is unreadable, as the direct adjudicator does, so their coverage is 54.17%. The two differ in adjudication rule, not in scoring convention: `direct_only` asserts a violation whenever the authorization surface reports no granting channel, regardless of the request outcome; `direct_observe` additionally requires the ordinary request to succeed. All 108 FP claims of `direct_only` fall on requests the server in fact denied. These are invalid violation claims without unauthorized-access effect. Strict false proofs, defined elsewhere as promoted claims on entitled access (*E*=allow), are a different subclass; both subclasses count as incorrect claims for the binary risk in this table. The last row is a clearly labelled binary reference that forces a negative verdict on those cells, and its misses are the price of that forcing, not a property of the method. Panel (b) splits v3's abstentions by reason: only the first row is the availability gate the method is named for.

method	coverage (%)	abstained	on violation	on normal	FP claims	misses	risk (%)
`raw`	100.00	0 (0.00)	0	0	120	3	21.35
v1 (existence filter)	86.98	75 (13.02)	15	60	60	3	12.57
v2 (existence + coverage)	89.58	60 (10.42)	0	60	60	3	12.21
v3 (state-calibrated)	60.42	228 (39.58)	108	120	0	3	0.86
`degrade_all`	42.19	333 (57.81)	213	120	0	3	1.23
`direct_only`	54.17	264 (45.83)	108	156	108	0	34.62
`direct_observe`	54.17	264 (45.83)	108	156	0	0	0.00
`direct_observe`, forced negative	100.00	0 (0.00)	0	0	0	108	18.75

(b) v3's abstentions by reason

reason	abstentions (on violation / on normal)	
surface unreadable	156 (108 / 48)	nothing to consult
surface readable, covering grant	72 (0 / 72)	a false proof converted into a non-decision

on the inputs the crawl reached. Neither point disputes the reported value or its labels.

Implementation robustness

An independently written implementation of the same predicates was compared on a subset fixed in advance; it bounds implementation accident only, and is reported with the confirmations in Section VII-D.

F. WHAT POOLING HID: THE ABLATION SPLIT BY ORACLE READABILITY

Table 5 pools two populations in which the oracle-consuming methods behave in opposite ways. The records differ in one respect that decides whether those methods can act at all: whether the introspection surface returns a reading. We therefore repeat the same comparison inside each stratum (Table 7), with the same methods and decision rule, and add two references that need no oracle.

Reconciling the representative grid with this partition

The 36/18 grid of Section IV is a single point of the switch space, not a miniature of the full partition. The gap closes additively and record by record: the two other exception policies add 108, the blind reading 162, S1–S2 144, and the two no-exception configurations 108, all unreadable. The blocks are disjoint and their union is the full 576 records; the readable count runs $36 \to 108 \to 216 \to 312 \to 312$.

Stratification variable

We stratify on the *effective* readability of the surface—the value the probe returns—not on the configuration switch: 108 records sit in configurations flagged `introspection=on` whose surface nevertheless returns nothing. The two strata partition all 576 records: 312 readable, 264 unreadable.

The readable stratum strengthens Finding 2

Splitting does not rescue the metamorphic step; it removes the comparison in which it looked best. On the 312 readable records `direct_observe` reaches recall 1.00 with 0 false proofs, against v2's 0.97 with 36 and v3's 0.97 with 0. The plain request is ahead on false proofs and not behind on detections, whereas the pooled table had let the family appear competitive. The comparison rests on Proposition 2, and `direct_observe` is not an independent method here, being built from the same two quantities that define the reference.

The unreadable stratum is the relation's own regime, and it is not empty

The 264 unreadable records contain 108 true violations and 156 correct cases, a base rate of 40.91%. Every oracle-bearing method detects none of them (0), because those methods have nothing to consult and abstain. The metamorphic family does not: v2 reaches all 108 true violations (recall 1.00) with 24 false proofs at precision 0.82, a lift of 2.000× over the base rate. `always_alarm` scores the base rate itself (0.41); `obs_success` reaches the *same* recall at precision 0.69, lift 1.692×. The net difference is therefore not in detections (0 additional true violations) but in false alarms: 24 fewer for the metamorphic family.

What the saving consists of

The attribution is one-sided and mechanical. Every record of the 24-alarm saving falls on S3, the legitimate share created *through the framework's own facility*: 24 false alarms for `obs_success` against 0 for v2 on the 48 records of that scenario. On S6, the same share created out of band, the two agree exactly (24 vs. 24), for the reason developed in Section VI-A. The incremental detection information the metamorphic step carries where no oracle exists is therefore exactly the information the crawl-derived model can see: it suppresses what was logged, and reproduces Finding 1 on what was not. That bounds Finding 2 rather than overturning it. Both references are deliberately weak and share an observ-

TABLE 7. The ablation of Table 5 re-run inside each stratum. The two panels are not rankings of the same population but two different regimes: in the readable stratum the oracle-bearing methods can act; in the unreadable stratum they cannot act and are structurally bound to zero detections. The two reference rows need no oracle and no crawl model: `always_alarm` is the trivial floor, `obs_success` alarms on the observed success of a plain request. All numbers are counts.

stratum	method	TP	FP	FN	TN	precision	recall	F1
readable (312)	`raw`	105	72	3	132	0.59	0.97	0.74
	v1 (existence filter)	96	36	12	168	0.73	0.89	0.80
	v2 (existence+coverage)	105	36	3	168	0.74	0.97	0.84
	v3 (state-calibrated)	105	0	3	204	1.00	0.97	0.99
	`direct_observe`	108	0	0	204	1.00	1.00	1.00
	`obs_success`	108	96	0	108	0.53	1.00	0.69
	`always_alarm`	108	204	0	0	0.35	1.00	0.51
unreadable (264)	`raw`	108	48	0	108	0.69	1.00	0.82
	v1 (existence filter)	102	24	6	132	0.81	0.94	0.87
	v2 (existence+coverage)	108	24	0	132	0.82	1.00	0.90
	v3 (state-calibrated)	0	0	108	156	—	0.00	—
	`direct_observe`	0	0	108	156	—	0.00	—
	`obs_success`	108	48	0	108	0.69	1.00	0.82
	`always_alarm`	108	156	0	0	0.41	1.00	0.58

able with the reference labelling, so only the differences are meant to be read.

V. REAL SYSTEMS: GITEA, GOGS, GITLAB, AND A SEEDED CONTROL

We repeated the essential comparison on real systems, choosing Gitea 1.22.6 for object-level sharing with per-object visibility, a machine-readable sharing surface, and a documented administrative grant path. Gitea enforced the expected decisions on the endpoints exercised here, so the experiment measures the false-proof side; the reachability model is reported under *both* readings—crawl-time snapshot and live execution-time state—so the reading moment is the explanatory variable. The experiment produces 96 cells, 48 per reading.

Grant surface

An endpoint is grant-capable if, without modifying the system, a request can move a (subject, object) pair from not-entitled to entitled. Of the 253 paths in the system's API description, 15 are grant-capable, 8 of those are drivable by a non-administrator—the out-of-band channel in its real form—and 2 revoke. The cross-vendor table reports a normalized census, which is why its Gitea figure (13) differs from the raw count here.

Is the authorization surface readable, and from where?

The counts below are different populations, not one denominator (Table 8). From the owner/administrator vantage the sharing surface is readable on 56 of 56 attempts: complete. From a black-box subject's own vantage it is readable on 0 of 40: never. An intermediate count of 16/56 comes only from subjects who happen to be owners or administrators. On 48 of 48 applicable cells the credential-substitution check confirmed that our model agrees with the system's behaviour (Fig. 3).

Is the write path verified by state rather than status?

A status code cannot establish that a write took effect, so the write path is checked by content: the subject creates a uniquely named file, the owner re-reads it byte-for-byte, and the file is deleted where the API allows. Across the 3 systems,

TABLE 8. Authorization-introspection readability protocol on Gitea. Each row is a distinct population and denominator, so the rows are not comparable.

vantage	attempts	readable
owner-vantage authorization-introspection surface	56	56
arbitrary subject (black box)	40	0
subject also owner/admin (representative grid)	56	16

120 of 120 cross-credential write attempts agree with the ledger, with 0 unauthorized writes, and no status code contradicted the content outcome (0). The owner control changed content in 24 of 24 attempts. On 2 of the 3 systems the site administrator could not write through the contents API although the ledger grants write; those 24 control rows are excluded from the denominator and reported as a modelling gap.

The mismatch is in time, not in visibility

The adjudication departs from the real authorization state on 10 of 48 cells under the crawl-time reading and 0 of the same cells live, with system state and tested relation held fixed. Under the crawl-time reading a bookkeeping adjudication produces 10 false proofs on the 25 authorized cells (rate 0.40), against 0 alarms live; v3 produces 0, and `direct_observe` emits 0. The unfair control `direct_only` raises 23 alarms because it ignores the request outcome, yet none is promoted (0), so alarm count and false-proof count are not the same quantity. Channel provenance of the authorized access is mostly not observable by the framework log (24 of 25, 0.96), a property of our topology rather than a field prevalence. Two snapshots cannot separate a stale model from a channel the crawl never listed; the transition experiment separates them.

An endogenous control

The most serious alternative explanation is that public objects are harder to enumerate—visibility, not time. Two objects that end equally public but became public at different moments control for this: the predicate reports unreachability on 5 of 6 cells only for the object made public after the crawl, and on 0 for the object public throughout, with the MR firing

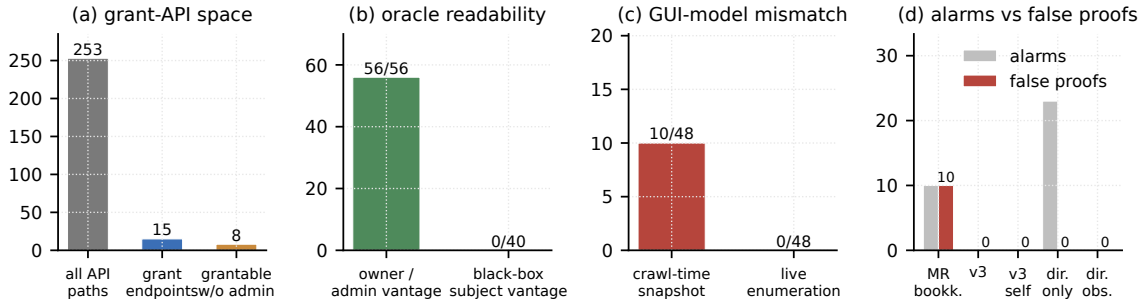


FIGURE 3. Gitea 1.22.6. (a) the grant-API space; (b) authorization-surface readability, complete from the owner/admin vantage (56/56) and never from a black-box subject's (0/40); (c) the departure from the real authorization state is present under the crawl-time reading (10/48) and absent live; (d) alarms versus false proofs per adjudicator. All counts are recomputed from the run records by the analysis pipeline.

correspondingly (5 vs. 0). Final visibility is identical, so the only manipulated explanatory variable is when the model was read.

A. EIGHT COMMITTED TRANSITIONS WITH A FROZEN MODEL, ON TWO PRODUCTS

We ran eight authorisation transitions on one fresh Gitea 1.22.6 instance and repeated the same eight, under the same frozen protocol (hash `d09d904707aa`) and ledger rules, on an independent GitLab CE 17.11.7 instance. The mechanisms differ per product, so the two runs share a protocol, not a topology, and we report them side by side rather than pooled.

Each transition is a closed loop: crawl every subject, serialise the per-subject reachable set, freeze it as M_0 and record its digest; commit one authorisation transaction; recrawl with the *same* credentials to obtain M_1; read the oracle from the owner vantage; issue the request and verify the effect by content; classify against the ledger at decision time. All 16 transactions committed, all 16 frozen M_0 digests were unchanged when re-checked immediately before probing, and all 16 cycles were version-consistent, over 6 gate checks (4 on GitLab). Across 712 cross-credential cells (280 on Gitea plus 432 on GitLab), E and Y agree in 712 cases (432 of the GitLab cells) under the closed-world default reading; Table 11 reconciles this with the literal *unknown* rule, under which the scorable set is the 171 E-allow cells and the agreement is 171/171.

The stale model produces 29 false proofs (13 on Gitea, 16 on GitLab), and a recrawl with the same credential removes 29 of them, leaving 0 still missed and 0 live false proofs (Table 10). The composition is informative rather than uniform, and the two products agree on its shape: one cell for the direct grant, one for the inherited-channel attachment, five and six when a repository becomes public, and six and eight when a user becomes site administrator—the cases a single-object probe is least likely to anticipate, because the granting event is not about the object. Revocations must not be scored as misses: the old model still lists 17 cells whose access was revoked (7 on Gitea, 10 on GitLab), and in every one the request is refused.

The multi-channel transition is where the products part company: on Gitea, after the team member is removed, `acme/t1` remains readable through a direct collaboration while `acme/t2` becomes unreadable, and the ledger records the remaining channel. On GitLab removing a member from a group also deletes that user's project-level membership, so we placed the alternative channel on a personal project and record the cascade as a product-semantics observation rather than a ledger rule (amendment A2).

The oracle column separates the products more sharply than the transition counts. On Gitea the owner vantage answers 224 of 280 cells (the rest are anonymous), but the subject's own vantage answers only 7—all after the site-admin grant. On GitLab the owner vantage answers 360 of 432, and the subject's own credential answers 360, because GitLab's member endpoint is readable by any project member. That readability is not soundness: the endpoint answers "is this principal a member", so it returns 404 for a public project a non-member may read and for a site administrator who may read everything. Measured against the ledger, the Gitea oracle disagrees on 0 cells and the GitLab oracle on 53, all of the latter in the public (44) or site-administrator (9) channel.

Table 9 assigns each cell to exactly one channel, by the strongest channel that grants it; Table 13 gives the same decomposition for Gogs, because a channel label is a property of the entitlement ledger, never of a probe response. Both tabulated products answer every direct-collaborator and group cell correctly, and Gitea also answers its public and site-administrator cells correctly. GitLab errs only by omission, on the two channels that are not membership, and never returns a false allow; a one-sided error is enough to manufacture a false proof.

Two properties of the run are what an external check can verify. The protocol was frozen before the first Gitea transition, and its digest is carried in both run records. For the amendments we do not rely on a self-reported clock: the archived pilot records only A1, the reported GitLab run carries the digests of both A1 and A2, and A2's `written_at`

TABLE 9. Oracle fidelity by granting channel, over the same transition cells as Table 10: a confusion matrix of the owner-vantage oracle against the ledger, per channel and per product. *allow* and *deny* count cells where the oracle agrees with the ledger; *f. allow* and *f. deny* are the two error directions; *unans.* counts cells where the surface returns no definite answer (anonymous readers). A false deny is an oracle that answers deny on a legitimately readable cell. Neither product produces a single false allow: the membership surface errs only by omission, and public and site-admin cells are exactly the channels that are not group or repository membership. The "no channel" row is the default-deny stratum: no grant of any kind is recorded for those cells, so the ledger denies them.

(a) Gitea 1.22.6

Channel	cells	allow	deny	f. allow	f. deny	unans.
site administrator	7	7	0	0	0	0
public visibility	44	35	0	0	0	9
direct collaborator	25	25	0	0	0	0
team / group	1	1	0	0	0	0
no channel	203	0	156	0	0	47
total	280	68	156	0	0	56

(b) GitLab CE 17.11.7

Channel	cells	allow	deny	f. allow	f. deny	unans.
site administrator	9	0	0	0	9	0
public visibility	53	0	0	0	44	9
direct collaborator	25	25	0	0	0	0
team / group	7	7	0	0	0	0
no channel	338	0	275	0	0	63
total	432	32	275	0	53	72

(02:32) is later than the run's last probe at 02:26:14—we retain the field as recorded and rely on the digest, with the discrepancy documented in the artifact. Inside a run, every cell carries its event sequence number, the harness version before and after, the M_0 and M_1 digests and wall-clock timestamps, so the ordering can be re-checked line by line. The residual 0 is a count on these surfaces, and the eight transitions are a designed set, so none of the counts is a field rate.

What a recrawl repairs, and what it cannot

The 29 repairs are not a general remedy, because a grant can be absent from the model for two causes. *Staleness*: the grant is visible through the subject's own interface, was made after the snapshot was frozen, and a fresh crawl with the same credential lists it again. *Coverage*: the grant is real and enforced, but the enumeration the crawl runs does not return the object, because the view that would return it belongs to another subject or to an administrative interface. Recrawling cannot repair a channel the crawl has no way to enumerate, and the cross-vendor run locates the residue: on Gogs 13 of 19 mismatching cells survive the live reading—5 on public visibility and 8 on the site-administrator vantage—and 5 of 11 bookkeeping false proofs survive, all 5 of them on public visibility. No team-derived or collaborator grant survives, because this product resolves team inheritance in the subject's own list. The live reading removes the staleness term and leaves the coverage term; where the same channels are represented in the subject's crawl, as on Gitea and GitLab, the live reading reaches zero.

That attribution is measured, not subtracted: in one instance, with the same subject, operation, surface and budget, and every grant made after the snapshot by another account, three channels ran in parallel—direct collaborator, team-derived, public repository (3 arms, 3 alarms). The recrawl repaired 2: the collaborator and team objects appear in the subject's list on the second pass. It left 1: the public repository, readable in the same probe as the other two but not returned by the enumeration the recrawl runs—the subject's own repository list, the organisation list, and the first page of an unfiltered search. A query that names that repository does return it (1 hit), so the boundary is the enumeration's page and filter rather than the system's visibility rules. What generalises is the mechanism rather than the ratio: a recrawl repairs exactly those grants whose granting channel the renewed crawl can enumerate.

B. CROSS-VENDOR REPLICATION OF THE SNAPSHOT DESIGN: GOGS 0.14.3 AND GITLAB 17.11.7

We repeated the same three-phase protocol and 48-cell design on two further systems (Table 12): **Gogs** 0.14.3, the project Gitea forked from in 2016 (census from its pinned route table, 80 paths), and **GitLab** 17.11.7 (census from the pinned running instance's routes, 1357 paths, since its OpenAPI document omits core grant endpoints).

The replication introduces a primary methodological variation: the authorization-introspection surface, which shows that *readability does not imply soundness*. Gitea's owner-vantage surface is readable and sound (agrees with effective access on all 48 cells). Gogs's collaborator-membership endpoint is equally readable (48/48) but unsound—wrong on 17 cells (35.4%), exactly the authorizations arising outside direct membership: site administration (8), public visibility (5), out-of-band grants (4). GitLab's `members/all/{u}` resolves inherited membership, but public visibility and site administration confer none: 18 false negatives (37.5%). What a surface cannot see is predictable from what it measures—why clause (2) of the admissibility condition cannot be automated from readability alone.

Table 13 locates those 17 errors on the ledger axis. Panel (a) groups the 48 cells by granting channel: the surface is correct on all 4 of the 4 direct-collaborator cells and all 27 cells with no recorded grant, answers *deny* on all 17 cells entitled through team membership (4), public visibility (5) or site administration (8), and never answers allow where the ledger denies (0). Panel (b) re-groups the same cells by mechanism:

TABLE 10. Temporal transitions on two products under the same frozen protocol. ***Stale FP***: the frozen pre-change model does not list a resource the subject may legitimately read, so the relation would report a false proof. ***Fixed***: a recrawl with the same credential lists it again. ***Missed***: the recrawl also fails to list it. Each product has its own mechanism per row (team attach ↔ project-group share; team member removal ↔ group member removal), its own cell count (280 on Gitea, 432 on GitLab) and its own decision-time ledger; no row is pooled across products.

	Gitea 1.22.6			GitLab CE 17.11.7		
Transition	Stale FP	Fixed	Missed	Stale FP	Fixed	Missed
direct grant	1	1	0	1	1	0
direct revoke	0	0	0	0	0	0
team attach	1	1	0	1	1	0
team member remove	0	0	0	0	0	0
private→public	5	5	0	6	6	0
public→private	0	0	0	0	0	0
role read→write	0	0	0	0	0	0
site admin grant	6	6	0	8	8	0
Total	13	13	0	16	16	0

TABLE 11. The same transition cells under the two ***unknown*** rules of Section III. The implementation scores every committed cell by closed-world default deny; the protocol's literal ***unknown*** policy marks modelled-no-channel cells E-unknown and excludes them from scoring. The rule changes the scorable set, the E classification and the E/Y denominator; it does not move the stale false proofs, all of which fall on E-allow cells inside the literal-known set. Table 1 gives the rates; this table gives the counts they are computed from, product by product.

	implementation (default deny)			literal protocol (E unknown)		
Product	scored	E allow	stale FP	scorable	E/Y agree	stale FP
Gitea 1.22.6	280	77	13	77	77	13
GitLab CE 17.11.7	432	94	16	94	94	16
Total	712	—	29	171	171	29

TABLE 12. Cross-vendor replication under the identical three-phase protocol (crawl, out-of-band authorization change, adjudicate) on Gitea 1.22.6, Gogs 0.14.3, and GitLab 17.11.7. Counts are per reading of the same 48-cell grid; the live reading is given in parentheses where both exist. The snapshot reading is the operating point of the published relations. Gitea and Gogs share lineage (fork, 2016); GitLab shares none. The team-derived row is read out of the per-system oracle false negatives by channel (Section V-B). The Gitea grant count here is the ***normalised*** M1 census (semantically equivalent routes merged, non-authorization grant surface excluded), and therefore differs from the raw route count reported in Section V; the two tables use different normalisations and are not in conflict.

	Gitea 1.22.6	Gogs 0.14.3	GitLab 17.11.7
API entries in pinned version (M1 denominator)	253	80	1357
Grant endpoints (M1)	13	10	18
of which usable without site admin	10	4	17
Owner-vantage oracle readable	48	48	48
oracle agrees with actual access	48	31	30
oracle wrong (false negatives)	0	17	18
oracle wrong: team-derived access missed	0	4	0
Subject-vantage oracle readable	8	35	48
Reachability mismatch, snapshot (live)	10 (0)	19 (13)	10 (0)
False proofs: bookkeeping alarm, snapshot (live)	10 (0)	11 (5)	10 (0)
False proofs: v3 (oracle-gated), snapshot (live)	0 (0)	8 (5)	5 (0)
False proofs: direct-only (oracle only)	0 (0)	17 (17)	18 (18)
False proofs: direct-observe (oracle ∧ request)	0 (0)	17 (17)	18 (18)

4, 5 and 8 cells, all wrong, against 4 and 27, all right. The blind spot belongs to the chosen surface, not to the API: the per-object permission suffix returns 404, the per-object team route 404 and the collaboration route the oracle consults 204, while the organisation team listing is reachable (200) but is not a per-object authorization surface. The live reading gives 31 agreements and 17 false denials. 1 cell carries two ledger rules and is counted once under the stronger rule, so the rows sum to 48 cells rather than the 49 recorded rules.

The consequence is measurable: on Gitea the surface-gated adjudicators produced no false proofs (0 v3, 0 direct-observe), while on Gogs they manufacture them (8 snapshot, 5 live, 17 direct-observe) and on GitLab again (5, 18), every v3 false proof falling on a cell whose authorization derives from site administration or public visibility (8 and 5 such cells on Gogs; 8 and 10 on GitLab). No admissible oracle was available from the tested surfaces on Gogs or GitLab, so the honest adjudicator abstains where those surfaces would license a promotion.

Bookkeeping alarms false-proof on legitimately granted access on every product (Gitea 10, Gogs 11, GitLab 10 cells) with the same out-of-band-dominant grant profile, and the reachability mismatch persists on all three (Gitea 10, Gogs 19, GitLab 10 snapshot). On Gogs the mismatch only partially resolves live (13 remain); on GitLab the live reading recovers everything (0 remain)—a second, vendor-specific failure of the snapshot premise, in the crawl model rather than the oracle.

C. A SEEDED POSITIVE CONTROL

The three real systems enforce the expected decisions, so the question remains: *when a real object-level flaw is present, does the same detector find it?* A *controlled authorization target* emulates the Gitea API contract the engine con-

TABLE 13. Oracle fidelity by granting channel on Gogs 0.14.3, over the same 48 snapshot cells as Table 12. Channel labels come from the entitlement ledger, which is built from the lab topology, the documented inheritance rules and the recorded out-of-band grant events, and never from a probe response; they are therefore available even though the product's own API exposes no per-object membership route that separates direct from inherited grants. *allow* and *deny* count cells where the owner-vantage introspection surface agrees with the ledger; *f. deny* is the surface answering deny on a cell the ledger allows; *unans.* counts cells with no definite answer. On this product the observed effect E coincides with the ledger entitlement on every cell, so the same table reads as $E \times O$. Panel (b) re-groups the same cells by the mechanism that produced the entitlement: the owner-vantage per-object collaboration route that the oracle consults represents direct collaborators only, so team membership, the repository visibility flag and site-administrator status are invisible to it. Whether some other endpoint in the same API could represent those grants is a separate question, and not the one a surface-gated adjudicator asks. One cell carries two ledger rules (site administrator and public visibility) and is counted once, under the stronger rule, so the rows sum to 48 cells rather than to the 49 recorded rules.

(a) by granting channel (ledger)

Channel	cells	allow	deny	f. deny	unans.
site administrator	8	0	0	8	0
public visibility	5	0	0	5	0
team / group	4	0	0	4	0
direct collaborator	4	4	0	0	0
no channel	27	0	27	0	0
total	48	4	27	17	0

(b) by granting mechanism (same cells)

Mechanism	cells	agree	f. allow	f. deny	f. deny (%)
direct collaborator (collaboration endpoint)	4	4	0	0	0.00
team membership (inherited)	4	0	0	4	100.00
repository visibility flag	5	0	0	5	100.00
site-administrator status	8	0	0	8	100.00
no grant recorded	27	27	0	0	0.00

TABLE 14. Positive control: detection of 3 seeded vulnerable instances (one fault pattern). TP/FN/FP/TN are counted against the known seed, not against a model.

method	TP	FN	FP	TN
`mr_bookkeeping`	3	0	2	43
`v3` (owner vantage)	3	0	0	45
`v3` (subject vantage)	3	0	0	45
`direct_only`	3	0	30	15
`direct_observe`	3	0	0	45
`direct_observe` (subject)	3	0	0	45

sumes, with a correct internal authorization model and 3 seeded vulnerable instances, all instantiating one fault pattern (a none-entitlement subject served the object at `GET /api/v1/repos/{owner}/{repo}`), differing in subject and object, not in fault location. Its introspection surface reports the *correct* `none` permission, so the defect is visible only as a discrepancy between oracle and request. We drove it with the same topology, mutation, crawl, introspection and adjudication code as above; ground truth comes from the seed list.

Table 14 is consistent with the negative result of the ablation: `direct_observe` and `v3` catch all 3 planted instances with zero false positives, while `direct_only` raises 30 alarms on cells where the surface says `none` and no access succeeds—alarms our strict definition does not promote. Where a surface that fully resolves the authorization question is available, the metamorphic step is not necessary for detection. The boundary is explicit: we seeded one BFLA defect (non-privileged bob succeeding at `PUT .../collaborators/{u}`, HTTP 204), which the read-oriented logic does not cover.

D. FOUR DEFECT CLASSES ON THE CONTROLLED TARGET

We seed four classes on the same target—the read bypass already in use with three instances, plus two instances each of three new classes. C1 is the read bypass. C2 is a write bypass: a subject whose true permission is `none` changes the object's metadata with `PATCH`. C3 is a faulty inheritance expansion: a member outside the team is served the team's repository, and the listing, the search results and the object read all agree that the subject may see it. C4 is an escalation: a subject that is neither owner nor administrator grants a third party access it should not have.

Per class—three instances of C1, two each of C2–C4—the read gate recovers 3, 0, 0 and 0; the channel-matched gate 3, 2, 0 and 0; the oracle-consuming adjudicator 3, 0, 2 and 2 on the read channel and all 9 on the channel-matched probe, and the channel-matched relation 5 of 9. False alarms on authorized cells of the same channel are 8/23, 6/16, 8/23 and 0/7 for the relation and zero for the oracle. C1 is the only class the read relation covers. C2 breaks a different channel: the relation asks whether the object is *served*, not whether it can be *changed*, so it is silent on both instances; moving the same relation to the channel the defect breaks recovers both. C3 and C4 are undetected end to end, because in both the subject already sees the object, so `cannotReachThroughGUI` is false and the antecedent does not hold. They stay in the denominator, and the antecedent's failure is the explanation, not an exclusion.

The oracle-consuming adjudicator fires on all 9 instances once the probe is channel-matched, and on 7 of 9 when confined to the read channel. False alarms are counted per channel, because a read cell cannot stand in for a write or a grant: 23 read cells, 16 write cells and 7 grant cells. The relation's bookkeeping variant alarms on 8 of 23, 6 of 16 and 0 of 7, and the oracle-based adjudicator on 0 of each. Write success is judged by the object's state changing, not by the status code. The oracle answers these recorded cases correctly, which is soundness on this target, not independence. The leave-one-out is reported per method: on the channel-matched adjudicator, disabling a class's fault removes that class's alarms and leaves the others in place, for all 4 classes;

TABLE 15. A publicly fixed defect (object-versus-path ownership). The verdict uses *E* from the ledger and *Y* from the owner's re-read, not the write status.

version	PATCH	re-read	verdict
1.20.5 (vulnerable)	200	changed	true positive
1.20.6 (fixed)	404	unchanged	no claim

the read relation provides the same evidence for 1 class and the channel-matched relation for 2.

E. A PUBLICLY FIXED DEFECT

We also ran one defect nobody here planted: Gitea 1.20.5 against 1.20.6, the release whose notes record a security fix titled "Fix comment permissions" (`#28213`, backported as `#28216`); the same defect is catalogued as CVE-2023-49946 for Forgejo. What is missing is object ownership rather than path permission: a caller entitled to the repository named in the URL can address an object belonging to a different, private repository. Each version runs as a fresh instance: alice owns a private repository with one issue comment, bob owns a public one, and bob sends `PATCH/api/v1/repos/bob/pub/issues/comments/{alice's comment id}` with a replacement body. On 1.20.5 the write succeeds (HTTP 200) and alice's own re-read returns the replacement body, so the effect is real and not a status-code artifact; on 1.20.6 the same request is refused (HTTP 404) and alice's re-read is unchanged (Table 15). The ledger gives bob no write channel on alice's repository (E = deny), so the vulnerable cell is a true positive and the fixed cell raises no alarm: 1 true positive and 0 false proofs across the pair. All 10 control writes behave as expected in both versions, and alice's restore returns the body to the original in 2 of 2 instances.

The relation is not executed on this pair, so the alarm recorded here is the E/Y gate's own verdict: the result measures the gate, not the relations' recall on a real defect, and it shows the gate issuing a correct violation claim and ceasing to issue it once the vendor fixes the defect.

VI. FINDINGS, ADMISSIBILITY, AND FALSIFICATION

Each finding is a boundary on what the published relations can establish about object-level authorization—*temporal*, *oracle* and *measurement*—carried by the studies of Sections IV–V. The confirmations they rest on are collected in Section VII.

A. FINDING 1 (RQ1): DETERMINISTIC FALSE PROOFS—THE TEMPORAL BOUNDARY

Taken as published—raw alarms, or alarms filtered by bookkeeping exceptions—the adjudication reports a legitimate access whose grant the crawl record does not contain as a finding of an established violation under the promotion policy P of Section III-0c, in the direction forced by the two-valued predicate encoding. The two studies supply the grant differently: the constructed study seeds it outside the framework's record, the temporal study makes it between the crawl and the request. On the constructed system the rate is 1.00 (8 of 8 alarms) against 0 for the same share created in band; on Gitea, 10 false proofs on 25 authorized cells (0.40) under the crawl-time reading, against 0 alarms live, with objects, entitlements and predicates held fixed. The direction is forced: the predicate has two representations, and a grant created outside the crawled interface has none but "cannot reach". The predecessor attributes its residual false alarms to crawler coverage [3]; we add that the failure is a function of *when* the model was read.

A finding about a pair is only as sharp as the half that is chosen. Table 16 applies four promotion policies to the same 29 alarms over the 712 scored transition cells. The alarm count is invariant—no policy changes what the relations return, and none recovers a violation—while the number of claims, and therefore of contradictable claims, moves. P1 promotes every alarm and is the reading Finding 1 quantifies: 29 claims, 29 contradicted, none confirmed. P0 reports alarms without making claims and yields no false proofs, but its 29 withheld alarms are the same phenomenon left unadjudicated. P2, the oracle-gated policy the state-calibrated variant implements, makes 13 claims, all 13 of them contradicted, on the product whose introspection surface returns false denies, while on the other it withholds 13 alarms. P3 promotes only where the ledger records a definite denial and returns zero false proofs by consulting the reference—a policy that cannot be wrong and cannot be a detector.

B. FINDING 2 (RQ2): NOTHING BEYOND AN ADMISSIBLE ORACLE

If the false proofs of Finding 1 are removed by consulting a readable introspection surface, it is the oracle that carries the work: for the authorization relations and scenarios evaluated here, once the oracle fully resolves the authorization proposition under test, replacing the entire metamorphic step with a single ordinary request—one that consumes the same readable oracle and access outcome—adds no adjudication information. The reason is partly algebraic: on the constructed grid the agreement is an identity, because the comparator is built from the same quantities that define the reference. The empirical burden is in the antecedent—that such an oracle is readable, sound and independent.

What is construction here, and what is empirical

`direct_observe` claims a violation exactly when the oracle answers *deny* and the ordinary request succeeds; the reference label calls a cell a violation exactly when the ledger answers *deny* and the content-level effect answers *allow*. Both rules are built from the same two quantities, so on the domain where the oracle agrees with the ledger and the request outcome with the content effect they *cannot* disagree: the agreement is an identity, a characterisation of two adjudication rules rather than a measurement of one technique against another. What is empirical is the domain—whether the oracle is readable, sound *and* admissible on a given deployment. Proposition 2 is stated conditionally for that reason.

TABLE 16. The same 29 alarms under four promotion policies, over the 712 scored transition cells of Table 10. An alarm *A* is the relation's own output and does not depend on the policy; a *claim* is a policy's promotion of *A* to `violation-proven`; a claim is *contradicted* (a false proof) when the ledger records the access as legitimate. *withheld* counts alarms the policy declines to promote. P0 and P3 both reach zero contradicted claims and neither is evidence of a better detector: P0 makes no claim at all, and P3 decides by consulting the reference ledger. P2 halves the count and leaves it entirely on the product whose introspection surface returns false denies.

policy	alarms	claims	confirmed	contradicted	withheld
P0 report the alarm, no claim	29	0	0	0	29
P1 promote every alarm	29	29	0	29	0
P2 oracle-gated (v3)	29	13	0	13	16
of which Gitea	13	0	0	0	13
of which GitLab	16	13	0	13	3
P3 ledger-gated	29	0	0	0	29

The direct adjudicator. Fix, before any run, the rule D that issues `violation-proven` for (o, s, a) exactly when the oracle O answers *deny* and the ordinary request R succeeds. D consumes O and R and nothing else; it is the comparator of the ablation in Section IV-C (`direct_observe`).

Lemma 1 (absorption)**.** Let a detector issue its verdict through a map $g(A, O, R)$, where A is its alarm and O, R as above. If, on the domain of cells considered, (i) A is a function $a(M, R)$ of the crawl-time model and the response only, and (ii) M is O-determined for the authorization proposition under test—that is, M carries no distinction the oracle does not already make—then there is a map g' with $g(a(M, R), O, R) = g'(O, R)$ on that domain.

Proof. By (i) the first argument of g is a function of (M, R); by (ii) M is a function of O on the domain; substituting gives $A = a'(O, R)$, so g factors through (O, R) as $g'(O, R) := g(a'(O, R), O, R)$. □

Proposition 2 (what the relation can add once the oracle is admitted)**.** For the two relations examined here, premise (i) holds by source inspection: their predicates read the crawl-time reachability model and the response, and nothing else (Section VII-B). Premise (ii) is an empirical condition on the recorded domain, and we do not assume it. Write y for the response bit, m for the model bit and o for the oracle bit; the shipped alarm is $A = y \wedge \neg m$, the direct adjudicator is $D = y \wedge \neg o$, and the promoted verdict of the state-calibrated method is $H = A \wedge \neg o$, which abstains where o is undefined. On the domain where o is defined:

(a) $A = D$ if and only if $y = 0$ or $m = o$; the two agree on every record whose request fails, and part exactly on the rows with $y = 1$ and $m \neq o$;
(b) $H \Rightarrow D$, because $A \Rightarrow y$ and the two share $\neg o$. The metamorphic step therefore cannot confirm a record that the direct consumer of (y, o) does not confirm; it can only abstain or repeat.

Part (b) is algebra and needs no premise about the model. Part (a) is informative only where $m = o$, and that equality is measured: a sound oracle does not make the crawl-time model sound. Row 7 of Table 17 holds 14 cells with $y = 1$, $m = 0$ and $o = 1$—the oracle answers correctly, the model is stale, and the relation raises a false proof that D does not. Premise (ii) therefore fails on the recorded domain: rows 7 and 8 share $O = \text{allow}$ but carry different M. What the proposition establishes is narrower and sufficient: once a definite oracle is available, promotion through the relation is a *subset* of direct adjudication.

The two shipped predicates read

$$A_{002} = \neg\text{sup}(B, A) \wedge \neg m \wedge \neg\text{err}(o_1) \wedge q(o_1, o_2), \quad (1)$$

$$A_{004} = y \wedge \neg m, \quad (2)$$

where err is the error predicate and q normalised response identity. On the domain evaluated here $\neg\text{sup} = 1$, $\text{err}(o_1) = 0$ and $q = y$ in all 312 records on which the second clause was computed, so both collapse to

$$A = y \wedge \neg m. \quad (3)$$

Equations (1)–(3) reproduce the recorded alarm of all 576 constructed records and all 712 temporal cells with no disagreement (`analysis/ingest_truth.py`). The three records on which the state-calibrated method and `direct_observe` differ are the same S5 write cell under the three exception settings, where the model lists the object for reading, the alarm stays silent, and the oracle reports the write denied—a loss for the relation, not a gain.

Reading the table

Table 17 evaluates both verdicts on every combination of the four variables, split by the ledger's entitlement. A and D agree in 530 of the 584 cells with a usable oracle and part only in the two rows where $Y = 1$ and $M \neq O$, which hold 54 cells. The parting is two phenomena needing different remedies: in the 40 cells of row 6 the model is right and the membership surface answers *deny* on a cell both entitled and readable, so D alone raises a false proof that a better oracle would remove; in the 14 cells of row 7 the model is stale and the oracle right, so the relation alone raises a false proof that a recrawl would remove. Row 5 shows why agreement is not a certificate: in 13 cells both surfaces answer *deny*, the ledger says the subject is entitled, the request succeeds, and both verdicts are false proofs. The rows without an oracle are not idle: 128 cells return no definite answer, D is undefined on all of them, and the relation still alarms in 2. Nor is the model term free: the constant variant finds 108 violations at 72 false proofs, the current-listing variant 105 at 48—24 fewer false proofs for 3 fewer detections, over 288 records each.

TABLE 17. The truth table of the two relations, with the temporal cells that fall in each row. *Y*: the ordinary request succeeds. *M*: the crawl-time model lists the object for the subject. *O*: the introspection surface's answer, an em dash meaning it returns no definite answer. *A*: the alarm of either relation; on this domain both reduce to $y \wedge \neg m$. *D*: the direct adjudicator, $y \wedge \neg o$, undefined in rows 9–12. H_{v3} (abbreviated *H* below) is the state-calibrated verdict $A \wedge \neg o$. It differs from the native promotion policy *P*, whose verdict is $H_P = A$. The state-calibrated verdict exists only where *O* is definite and is therefore a subset of *D* by construction ($H{=}1 \Rightarrow D{=}1$); on the definite-oracle rows it agrees with *D* except in row 6, where *D* fires and *H* does not. The *A*/*D* columns are the two measured surfaces, and the disagreement count below is an *A*/*D* count—it must not be restated as an *H*/*D* count. Rows 9–12 leave *H* undefined like *D*. The count columns split each row by the ledger's entitlement *E*, so every row states its truth conditions for both values of *E*; a dot is an unrealised combination, and in the verdict column the first term reads *E*=deny and the second *E*=allow. Where *D* is defined the two verdicts coincide whenever *Y*=deny or *M*=*O*, and part only in the two rows with *Y*=allow and $M \neq O$—54 of the 584 readable cells. Those two rows are the two failure mechanisms that Section V-B keeps apart: in row 6 the model is right and the oracle wrong, so *D* alone raises a false proof (*A*=0, *D*=1, *H*=0); in row 7 the model is stale and the oracle right, so the relation alone does (*A*=1, *D*=0, *H*=0: the promotion suppresses the alarm, which is why row 7 must not be read as a disagreement between *H* and *D*). Row 5 is the case neither explains—both surfaces answer *deny* on a cell the ledger allows, both verdicts are false proofs, and their agreement certifies nothing.

Y	*M*	*O*	*A*	*D*	H_{v3}	verdict at the ledger	*E*=deny	*E*=allow
deny	blocked	deny	0	0	0	no access	416	·
deny	reach	deny	0	0	0	no access	15	·
deny	blocked	allow	0	0	0	no access	·	·
deny	reach	allow	0	0	0	no access	·	·
allow	blocked	deny	1	1	1	true positive / false proof	·	13
allow	reach	deny	0	1	0	miss / false proof by *D*	·	40
allow	blocked	allow	1	0	0	true positive / false proof	·	14
allow	reach	allow	0	0	0	miss / correct silence	·	86
deny	blocked	—	0	—	—	no oracle; no access	108	·
deny	reach	—	0	—	—	no oracle; no access	2	·
allow	blocked	—	1	—	—	no oracle; alarm	·	2
allow	reach	—	0	—	—	no oracle; miss / silence	·	16

The shipped code departs from Equations (1)–(3) in three places, none of which changes the algebra: the model term is a store fixed at crawl time; the error predicate is a status code, so a denial served with 200 makes *y* wrong; and response identity is not authorization identity. The proposition is falsifiable in one way: exhibit an observable Z, *other than the model and the response*, that a relation consumes and that is decision-relevant. It concerns *adjudication* only: a relation may still improve input generation or search-space reduction.

Scope, quantified

Section IV-F re-runs the comparison inside each stratum of effective oracle readability (Table 7). Where the oracle is readable (312 records) the plain request carries fewer false proofs (0 vs. 36) and no fewer detections. Where it is not (264 records) the antecedent fails, so the family is compared against oracle-free references: it retains an advantage over a trivial floor and over a single uninterpreted request (2.000× vs. 1.692× over the base rate), coextensive with what the crawl-derived model can see. Evidence: Table 5—0 false proofs vs. 0 (a tie at zero, not a win) over the 216 true-violation records, at recall 0.50 vs. 0.49, the two disagreeing on 3 and `direct_observe` right on every one. Gitea reproduces the zero-false-proof result (0 vs. 0); the control is *constructed*, not discovered, so the tie is the empirical face of the necessary condition.

C. FINDING 3 (RQ3): A POOLED SPECIFICITY CANNOT REGISTER A CLASS IT DID NOT EXERCISE

The reported 99.81% is a specificity over follow-up inputs, $\mathrm{TN}/(\mathrm{TN}+\mathrm{FP})$*, not a false-alarm share of all generated inputs; in that evaluation the MT failures were manually verified, so the labels are not the crawler's own. Its denominator cannot, on its own, provide scenario-level resolution for a failure confined to a class, and the inputs it counts are the ones the crawl could enumerate.* With $\mathrm{FN} = 0$ the figure is $1 - \mathrm{FP}/(F - \mathrm{TP})$, so a failure uniform across a class contributes only as many false positives as that class emits negative inputs and is diluted by the rest of the run (Proposition 1; Corollary 1). A class of legitimate access the crawl never reaches contributes no negative inputs, so the ratio is silent about it. Nor does the report fix the weights: if a failing class supplies a share w of the negative inputs the figure moves only between 1 and $1 - w$.

Fix a run ω and a method D; let $n_i(D,\omega)$ be the number of negative inputs D generates for scenario i and $f_i(D,\omega)$ the false proofs among them, so $S(D,\omega) = 1 - \sum_i f_i(D,\omega) \big/ \sum_i n_i(D,\omega)$ where the denominator is positive. The identity holds by counting on that run's records. For runs whose negative-input total stays positive $S_k \to 1$ iff the pooled false-proof share tends to zero, so a finite-sample $S < 1$ says nothing about the limit: holding the false-proof count at one while the total grows gives $S_k = 1 - 1/k \to 1$. Both cases coexist with an unexercised failing class.

D. COMPETING BASELINES: WHAT A SUCCEEDING REQUEST CANNOT SEPARATE

We ran three conventional differential rules on the same 42 cells (six configurations crossed with S1–S7), each consuming one request as the owner and one as the actor (Table 18). *B1 status-only* alarms whenever the replayed request returns a non-error status. *B2 equality replay* alarms only when the owner's request succeeded and the actor received byte-identical output. *B3* suppresses every cell covered by the tester's own share log. The fourth row is MR-002 with the v3 adjudication.

The cells contain 15 true violations, 12 legitimate accesses through a grant, and 15 denials. B1 and B2 claim on all 27 allowed cells: 15 true positives and 12 false proofs, a precision of 55.56%; the equality test adds nothing to the

status test, because every allowed access returns the owner's bytes. B3 removes the six in-band grants and still issues 6 false proofs, all on S6. Ours issues 10 true positives and no false proof (precision 100.00%), at the cost of 17 abstentions. The baselines find all fifteen violations (100.00% total recall), whereas ours confirms ten—those whose surface is readable—and declines the other five. Over all 15 violation cells total recall is 66.67% and coverage falls to 59.52%. A succeeding request is compatible with both a grant and a defect, so a rule that treats it as proof must either ignore grants (B1, B2) or see only the grants it created (B3). The out-of-band grant is the residual: on the Gitea snapshot the equality rule fires on all 25 legitimately allowed cells, and the in-band log still leaves 24. The budget is matched at the probe pair and only there—2 requests per cell for every method, 84 shared probes, and 111 requests for ours, 27 of them extra. We report no per-method timing: every method is scored from the same recorded responses in the same process.

The same comparison at a matched total budget

Table 19 replaces the probe-pair match with a total-request ceiling per cell on the same 42 cells. At a ceiling of two only the owner/actor pair fits: where the relation does not trigger our method passes (15 cells, recorded as true negatives); on the 27 cells where it does trigger, no request is left to read the surface, so it abstains. At a ceiling of three each method may spend one further observation declared in advance—ours reads the authorization surface, B3r enumerates the actor's visible objects—while B1–B3 spend nothing extra. Under that ceiling our method does not dominate: B3r reaches 14 of 15 violations with 4 false proofs, ours 10 with 0 and 17 abstentions (it passes the relation-free 15 cells), spending 111 of the 42×3 requests allowed. What survives a matched budget is the trade the paper describes—higher precision on the claims actually made, paid for in coverage—not a detection advantage.

E. ADMISSIBILITY, AND FALSIFICATION

Three properties must be kept apart. A surface is *readable* when its answer can be obtained from the vantage in question; *sound* when its answer equals the true authorization state; and *admissible*, for promotion to `violation-proven`, when (1) it is readable *and* (2) one can give a reason that it is not of the same kind as the defect under test—not itself a reachability artifact whose blind spot correlates with the blindness tested. Readability neither implies soundness nor confers admissibility: the owner-vantage surface is readable on every cell in each system (48 Gitea, 48 Gogs, 48 GitLab), yet sound on Gitea alone—it agrees with effective access on 48, 31 and 30 of those cells. The cross-vendor systems are therefore not cases of "no oracle": a readable membership surface exists on both, but it is unsound for the authorization channels exercised here, so it fails clause (2) (Gogs and GitLab convert the blind spots into 8 and 5 false proofs). Where clause (2) holds, an analyst who already has an admissible oracle should consume it directly (Proposition 2(b)). Where it fails—the case on every system studied here—dropping the relation for its direct counterpart is the wrong move: on Gogs 0.14.3 and GitLab 17.11.7 the direct adjudicator produces 17 and 18 false proofs against 8 and 5 for the oracle-gated relation (Table 12).

Clause (2) needs an operational procedure—defect classes, four evidence levels, an ordered *admit*/*reject*/*unknown* rule—fixed before the multi-class and equal-budget runs of Sections V-D and VI-D, and applied both to them and, conservatively, to the archived experiments. It is newer than the frozen temporal protocol and we do not backdate it. Per defect class the rule enumerates the path producing the enforcement answer y and the path producing the oracle answer o: *admit* if neither contains the other's decision point nor shares a component whose failure could co-occur with the tested defect, *reject* if such sharing is found, *unknown* if either path cannot be enumerated. On all three products the route the oracle consults and the routes that decide the request are answered by the same process under the same permission model (Gitea 1.22.6, Gogs 0.14.3 with its direct-collaborator-only per-object route, 404 for the permission suffix that would expose the rest, GitLab 17.11.7 with `members/all`), so the verdict is *reject*—at the API level, and also on the controlled target where the source is readable. Two revisions of the register, logged in place and without changing a verdict, corrected five of those grounds. Three epistemic statuses stay apart throughout: the diagnostic O/H readings, the classification of every promoted verdict against E and Y, and independence-certified proofs of previously unknown defects, which the rule refuses here.

Each finding would fall as follows: Finding 1, if the predicates were defined over live rather than recorded reachability; Finding 2, by an authorization MR needing an observable that a succeeding request plus a readable oracle cannot produce; Finding 3, by a specificity in scenario units that still fails to separate the configurations studied here. Table 20 states, for each research question, what is assumed, what was measured, and what the result does not cover.

VII. SUPPORT EVIDENCE

Nothing here is a further finding. It holds what the three findings rest on: the semantics of the two governing predicates read at source (Section VII-B), an execution of the published relations on a real system (Section VII-C), and the robustness of our numbers to a second implementation (Section VII-D).

A. THE ENUMERATION, CELL BY CELL

Split by relation, the 576 records fall 312 to MR-002 and 264 to MR-004; split by effective readability, 312 are readable and 264 are not. Section IV-F uses the second partition, and the cell-by-cell census ships with the artifact.

B. CONFIRMATION AT SOURCE

Definitions can be read generously, so we checked the predicates at source in the public library [2]: it ships 23 relations,

TABLE 18. Competing baselines on the same 42 cells and the same probe pair. The cells comprise 15 true violations, 12 legitimate accesses through a grant (in-band or out-of-band), and 15 denials. Every method consumes one owner request and one actor request (84 shared probes); ours issues one additional read of the authorization surface on the 27 cells where a relation alarms, and none where it does not (0.64 per cell on average, 27 in total, 111 against the baselines' 84), so total requests and visible information are not matched. A false proof is a cell with E = allow that the method still reports as a proven violation. Recall is reported twice because the two figures differ exactly where a method abstains: *decided* counts only the cells the method ruled on, *all* keeps every violation cell in the denominator.

Method	Decision rule	Claims	TP	False proofs	Abstain	Coverage (%)	Recall: decided / all (%)
B1 status-only	alarm iff the actor's request returns non-error	27	15	12	0	100.00	100.00 / 100.00
B2 equality replay	alarm iff the owner succeeded and the actor received byte-identical output	27	15	12	0	100.00	100.00 / 100.00
B3 B2 + in-band log	suppress cells covered by the tester's own share log	21	15	6	0	100.00	100.00 / 100.00
Ours: MR-002 + v3	consult the SUT authorization surface; abstain if unreadable	10	10	0	17	59.52	100.00 / 66.67

TABLE 19. Equal-budget comparison on the same 42 cells and the same credentials. *cap* is the total request budget per cell. At cap 2 only the owner/actor probe pair fits: our method keeps its own order—it passes on the 15 cells where the relation does not trigger, reading no surface, and abstains on the rest, where the relation does trigger and no request is left to read the surface. At cap 3 each method may spend one further observation of its own choosing—ours reads the authorization surface, B3r enumerates the actor's visible objects—while B1–B3 spend nothing extra and are reported as under budget. No method is given padding requests. Ours therefore does not dominate: at the same ceiling B3r reaches 14 of 15 with 4 false proofs, ours 10 with 0 and 17 abstentions, and where the surface is unreadable ours refuses to claim rather than guessing. Ours spends 111 of the 42×3 requests the ceiling allows because the 15 untriggered cells never read the surface—not because an abstaining cell is free: it still pays for its probe pair, and for the surface read whenever the relation triggers under a ceiling that reaches it.

cap	method	TP	FN	FP	TN	abstain	requests
2	B1 status-only	15	0	12	15	0	84
2	B2 diff-equal	15	0	12	15	0	84
2	B3 diff+in-band	15	0	6	21	0	84
2	ours (MR + v3)	0	0	0	15	27	84
3	B1 status-only	15	0	12	15	0	84
3	B2 diff-equal	15	0	12	15	0	84
3	B3 diff+in-band	15	0	6	21	0	84
3	B3r diff+in-band+reach	14	1	4	23	0	126
3	ours (MR + v3)	10	0	0	15	17	111

TABLE 20. Evidence to claim. Each row names the assumption the claim needs, the artefact that carries it, and the boundary that remains. The last row is not a research question: it checks the adjudication gate used throughout, and is not evidence about detection effectiveness.

RQ	Claim	Assumption it needs	Evidence	What it does not cover
1	A crawl-time reading reports an out-of-band grant of legitimate access as a proven violation.	The grant occurs after the crawl, is recorded in the ledger, and is reachable through a channel the crawl does not enumerate.	576-record constructed study; 16 committed transitions on Gitea and GitLab, 29 stale false proofs, all 29 removed by a same-credential recrawl.	Two relations of one reachability-gated family; designed transitions, not a sample of deployment changes; no residual miss is a completeness claim.
2	Once a readable, sound oracle is supplied, the relation adds no observed adjudication benefit over the direct consumer of the same oracle and outcome.	The oracle is readable and sound on the cells scored; the relation consumes no decision-relevant observable outside (O, R).	Paired ablation on 216 true-violation records, disagreeing on 3; channel-level oracle fidelity over 712 cells.	Conditional on the antecedent: the unreadable stratum has no oracle to consult, and the oracle shares its implementation with enforcement.
3	The published specificity is an input-level figure whose denominator cannot, on its own, expose a class-uniform failure.	Scenario classes emit comparable numbers of inputs, so a uniform failure is diluted by unaffected inputs; the class shares are not reported, so the dilution is bounded rather than estimated.	Proposition 1 and its corollary; sensitivity of the example to the assumed inputs per case.	The original per-case denominators are unavailable, so the published figure is diagnosed rather than re-estimated.
Gate	An independent entitlement ledger plus a content-level effect observation reports a real defect as a true positive and stays silent on its fix.	The ledger is complete for the tested object and the effect is read back by the owner, not inferred from a status code.	Gitea 1.20.5 versus 1.20.6: 1 true positive, 0 false proofs, 0 alarms on the fixed build, 10 control probes as expected.	One publicly fixed defect found and supplied by us; this is a check of the gate, not a recall estimate.

11 authorization relations, 8 gating on the two predicates analyzed. Four implementation-level properties are not made explicit in the main presentation: (1) *the freeze is hard-coded*—`cannotReachThroughGUI` is set-membership over the crawl inputs, issues no request, and is cached in a field written once and never invalidated; (2) *the clause that could have caught it is inert*—the "absent across all users" set is built by stopping each recorded sequence at the first login action, so it is always empty; (3) *missing data defaults to an alarm*—with an empty or null crawl argument the predicate returns "cannot reach", so an empty crawl *satisfies* the precondition, which is the mechanism behind Finding 1's deterministic direction; (4) *the role relation is configuration, not state*—the supervisor guard reads a static configuration file, so it cannot reflect a later grant.

C. EXECUTING THE PUBLISHED RELATIONS

The library ships relations but no execution environment; we supplied the system configuration and crawl snapshot and ran them—source unmodified, build configuration (`pom.xml`)

and an external harness added (1 changed upstream file, 0 changed Java sources, two added classes totalling 612 lines)—against Gitea 1.22.6. 10 of 11 authorization relations executed; 2 returned failures, 6 alarms in all—the engine's own `getFailures()` value, with 270 HTTP requests attributable to the observing relations. The claim is scoped to the *reachability-gated subset that fires here*: one relation aborts on an engine-side defect, most issue no request because their own guards are unsatisfied, and the content predicate is never invoked.

The load-bearing cell: the subject bob holds a legitimate entitlement to alice/s1, granted out of band after the crawl. There the published `cannotReachThroughGUI` answers "cannot reach"—the answer it is *defined* to give, since the subject's crawl record has no entry for the repository; across the run it answered so 1149 of 7321 invocations (15.7%), 5 on this cell, and 2 relations (OTG_AUTHZ_002, OTG_AUTHZ_002c) report a failure there. The access is authorized: all 3 granted cells were re-probed, and the grantee's response hashes identically to the owner's while a third account is refused. The 6 alarms fall on 3 cells, all post-crawl grants, none on a non-entitled object. The reproduced false proof is therefore that of the reachability-gated subset that fires.

Which stage of that claim does the native run establish?

Only the third of the three stages defined in Section III-0c is a security finding. The *native alarm A* is the engine's own return: `getFailures()` is appended by `MR.fail()` (`MR.java`:909), which logs `FAILURE` with the failing input pair and asserts nothing about authorization. The promotion policy P maps every fired `OTG_AUTHZ` relation to "a violation was observed", so $H = 6$ holds by construction. All of it is in the independent truth T: 0 violations are confirmed and 6 of the 6 promoted alarms are contradicted, on 3 cells—in each case a subject with a post-crawl grant, a ledger that records the entitlement, and responder bytes identical to the owner's while a third account is refused with 404.

D. IMPLEMENTATION ROBUSTNESS

An independently written implementation of the same predicates was compared with the main one on a subset fixed before the comparison, not sampled after it: the in-band scenarios on which a second reading is defined, holding the exception-policy variant at the representative level and the reachability reading at the one each relation uses. This yields 30 (configuration, scenario) points, 54 relation comparisons in all, on 54 of which the two agreed (1.00). It is a fixed enumeration, so no interval is attached.

VIII. THREATS TO VALIDITY

Source versus execution

We read the predicates at source (built under JDK 8 and Maven; Section VII-B) and then executed the relations, unmodified apart from the build configuration and our harness (Section VII-C). The harness substitutes an HTTP observation channel—exact for the reachability predicate, not exercised for the content predicate, whose invocation count we report rather than claim cleanliness we did not demonstrate. The run converts one class of claim, that a false proof is produced by the *published* relations on a real system under the promotion policy P; it does not convert the rest: the scenario/configuration and cell studies remain statements about our instantiation, and we report no sensitivity or specificity attributable to MST-wi.

Metric and statistics

Records within a design cell are not independent and the records are a fixed enumeration, so we report counts and rates rather than inferential intervals, and no result rests on statistical significance. The RQ2 ablation is paired and is reported as a discordance count. Finding 3 is a statement about what the published denominator pools over, not about the value of any specificity figure: no conclusion here depends on it.

Two MRs, one family

We instantiated 2 MRs of one family—relations gated on the absence of evidence. The native run lifts the count to 10 of 11 catalog authorization relations but not the family: only the reachability-gated ones alarm in our run. Widening the family requires relations gated otherwise, which is a different experiment.

Real systems

Three systems (one lineage pair, one independent), chosen for accessibility, none vulnerable on the tested surface, so only the false-proof side is measured, within one domain, the git forge. Channel proportions are topology design parameters, not field prevalence, and three systems tested once each are still not a population.

Stratification and the references used in it

The split of Section IV-F is on whether the introspection surface returned a reading for the record, not on the configuration switch: 108 records sit in configurations that enable introspection but whose probe returns nothing. The two oracle-free references there are deliberately weak—one unconditional, one guessing from a single request—because the point is to size the metamorphic step against methods that need nothing, not to identify the best oracle-free detector; a differential prober across accounts consumes more than those references do and is instantiated in Section VI-D instead. Both share an observable with the labelling rule, as `direct_observe` does, so only the differences against them are meant to be read.

IX. RELATED WORK

Metamorphic security testing

Metamorphic testing was brought to web security by Mai et al. [3], who cast security test patterns—authentication, authorization, session—as metamorphic relations so that faults can be exposed without an oracle. The line matured into MST-wi [2], whose 76 system-agnostic relations over a Web-specific domain language are the object of our study. We

TABLE 21. The published authorization relations executed against Gitea 1.22.6. Each row is the applicability chain from the catalogue to the outcome: *status* is the engine's per-relation result (10 of 11 evaluated, 1 aborted on an engine-side defect), *alarms* its own `getFailures()` return (2 relations, 6 in all), and *requests* (270 total) attributed to the relation observing them first.

MR	status	alarms	requests
OTG_AUTHZ_001	executed	0	0
OTG_AUTHZ_001b	executed	0	0
OTG_AUTHZ_001b2	executed	0	0
OTG_AUTHZ_002	executed	3	270
OTG_AUTHZ_002a	executed	0	0
OTG_AUTHZ_002b	executed	0	0
OTG_AUTHZ_002c	executed	3	0
OTG_AUTHZ_002d	executed	0	0
OTG_AUTHZ_002e	error	—	—
OTG_AUTHZ_003	executed	0	0
OTG_AUTHZ_004	executed	0	0

take the technique's own admission that the role-to-resource mapping is "not always available" at face value and follow it to its consequence for what the adjudication can decide. A relation whose guard reads a crawl-time model is sound in the false-proof direction whenever the model does not understate the subject's reachability; equality with the authorization state is a *sufficient* condition for that, not a necessary one—a model that overstates reachability turns a legitimate access into silence rather than into a false proof. That equality is exactly what the technique cannot check for itself.

Detecting broken access control

A large literature detects access-control flaws directly, and these techniques differ in the oracle they rely on. Black-box probing drives requests with and without another subject's rights and looks for a missing refusal [5], [6], a line running back to AUTHSCOPE [7] and extended to deployed systems under ethical constraints [8]; where an API description is available, the oracle itself can be synthesised [9]. A white-box line recovers the *enforced* policy from the implementation—by reverse-engineering RBAC policies [10], by statically extracting authorization policies [11] or database access-control rules [12], or by locating the permission checks themselves [13]—and flags violations against that recovered model: BOLA in database-backed applications [14], missing-owner-check and function-level flaws in Java web applications [15], [16], access-control flaws across Web applications [17], and privilege escalation in polyglot microservices [18]. Specification-driven API fuzzers generate the requests well but inherit the same oracle difficulty [19]–[21]; adding explicit access-policy violation checks [22] moves them toward the detection problem without removing the need for a ground truth, and a recent turn delegates generation or probing to language models [23]–[25]. A concurrent line reframes BOLA under a zero-trust model, in which each request must carry an explicit authorization decision rather than one inferred from endpoint reachability [26]; our question sits at that decision level. Each of these papers reports detections against an oracle it holds privately—a policy, a seeded defect, a differential response—and none is our comparison class, because we do not measure detection rates.

What this paper adds is three claims about one published adjudication, each stated with the check that would count against it. Two of the ingredients are already known—that a crawl-time model can go stale, and that an authorization oracle can be unsound; the claims differ in that the first failure is deterministic and attributable to a representation choice, the second is bounded by an explicit equivalence, and the third is a property of the published metric's denominator. First, a gate that reads a crawl-time model misreports a grant the record does not contain, deterministically (Finding 1). Re-reading the surface repairs that misreport only when the grant is one the surface would have shown on a second reading: across the two products a same-credential recrawl removes 29 of 29 stale false proofs, and on Gogs 13 of 19 mismatching cells persist under the live reading. Second, once a readable, sound oracle is supplied, the promoted verdict is a subset of direct adjudication on the evaluated domain (Proposition 2(b)); the check that would count against it is a record the relation confirms and the direct consumer does not. Third, the published specificity's denominator is pooled over the inputs a crawl could enumerate, so a failure confined to a class outside that set does not move it (Proposition 1). The three claims, the position each extends, what each adds and the check or counterexample attached to each are collected in Table 22.

Direct authorization oracles, and what is at stake here

Comparing responses across principals or privilege states is an established way to obtain an authorization oracle, and the black-box and white-box lines above are instances of it. We propose no new direct oracle, and we do not claim that direct authorization comparison is novel. Our subject is a different question: what an existing oracle-substituting technique *decides* when it substitutes reachability for entitlement. The direct oracle enters our design in one role—as the fair, same-observable control that a necessity claim about the metamorphic step must face.

LLMs and the oracle problem

LLMs are used for vulnerability detection and specification inference [27]–[30], including lightweight models over hybrid code features [31] and federated, explainable detectors [32]; we use none in the detection loop.

Measurement and evaluation

Our contribution is finally a measurement, and it connects to work on the quality of the artifacts security studies are built

TABLE 22. Novelty ledger: each claim of this paper beside the prior position it extends, the specific increment, and the check that would count against it. Rows 1 and 4 restate ingredients that are already known and are marked as such; the increment is in what is done with them, not in their discovery. For rows 1, 3 and 4 the last column names an observation that would refute the claim. The policy row is a decomposition rather than a proposition about the world, so its entry is the reproduction that would have to fail, not a counterexample.

claim	prior position	what this paper adds	check or counterexample
A legitimate access is reported as a proven violation	residual false alarms are attributed to crawler coverage [3]; the role-to-resource mapping is "not always available" [2]. *Known.*	The failure is deterministic and direction-fixed by which of the predicate's two representations a grant fits; the governing variable is *when* the model was read, not what was visible; and staleness (repaired by recrawling) is separated from channel blindness (not repaired)	A grant outside the recorded model that raises no alarm, or an alarm whose direction reverses
An alarm is a violation	The catalogue's purpose invites the reading; its code performs no verdict	The promotion policy is stated as a parameter, and four policies are quantified on the same alarms, separating the technique's output from the adjudication layered on it (Table 16)	Replay the declared policies on identical alarms and independent labels; the reported policy sensitivity is unsupported if the reproduced claim counts do not differ as reported
The metamorphic step adds adjudication value once an oracle is readable	Not posed; oracle-free detection is the design premise	Promotion is a subset of direct adjudication on the domain where oracle and ledger coincide, with the antecedent made operational as an admission test that refuses every system studied here	A record the relation confirms and a direct consumer of the same oracle and outcome does not
A pooled specificity supports scenario-level resolution	Specificity is reported as an aggregate over follow-up inputs [2]. *Known to be a corpus statistic.*	The denominator argument: a class the crawl never reached contributes no negative input, so the figure is silent about that class by construction (Proposition 1)	A failure confined to a class the crawl never enumerated that nevertheless moves the reported input-unit specificity

on—for instance the observation that vulnerability datasets are frequently mislabeled or otherwise unfit for the claims drawn from them [4]. The artifact we report is of the same family but inside a study rather than a dataset: a specificity figure whose *denominator* is pooled over the inputs a crawl could enumerate, so it cannot register a failure confined to a class that crawl never reached, while the adjudication that consumes the same crawl can.

X. CONCLUSION

For object-level authorization, MST-wi's predicates do not know whether the acting subject is entitled to the object; they know what the interface, as crawled, permitted. Instantiating them as defined produced three findings (Section VI).

Finding 1 (temporal)

As designed, the adjudication false-proofs on legitimately granted access, deterministically (1.00 constructed; 0.40 Gitea), because an out-of-band grant fits only the wrong one of the predicate's two representations. The alarm is the technique's own output; the count of proven violations is a policy's, and the same 29 alarms yield from 0 to 29 contradicted claims across the four policies of Table 16. On the real system the mismatch appears only under the crawl-time reading (10/48, vs. 0 live), with an endogenous control holding final visibility fixed. Under the same frozen protocol—whose two amendments are named and digest-bound in the second run's record—GitLab CE 17.11.7 commits 8 transitions over 432 cells: 16 stale false proofs, all 16 removed by a recrawl with the same credentials, 0 live false proofs and 0 residual misses. Pooled with Gitea this is 712 cells, 712 agreements, 29 stale false proofs and 29 fixes.

Finding 2 (oracle)

The natural repair—a readable, sound oracle that fully resolves the proposition under test—works, and in that regime the evaluated metamorphic step provides no incremental adjudication over a direct consumer of the same oracle and outcome. On the constructed grid this is an identity rather than an effect, so the matched counts (false proofs 0 vs. 0; disagreement on 3 of the 216 true-violation records) characterise two rules rather than measure one technique against another. The empirical content is the antecedent—readable, sound *and* admissible—and it fails on every system studied here. Where it fails, substituting the direct adjudicator is worse: 17 against 8 on Gogs and 18 against 5 on GitLab (Table 12).

Finding 3 (measurement)

The reported specificity of 99.81% is $\mathrm{TN}/(\mathrm{TN}+\mathrm{FP})$ over the follow-up inputs the crawl could enumerate, with manually verified labels, and that denominator cannot register a failure confined to a class the run did not exercise.

Reading the implementation confirmed the semantics and added the mechanism—the crawl model is frozen by a cache never invalidated, and a missing crawl defaults to "unreachable"—and the positive control closes the loop: on a seeded vulnerable target the surface-gated adjudicator detects all planted instances of the fault pattern it was built for with zero false positives, and on four seeded defect classes it detects the read bypass and nothing else. The entitlement/effect gate is in turn checked against a real defect: on a publicly fixed object-versus-path ownership flaw, Gitea 1.20.5 yields 1 true positive with 0 false proofs, and the fixed 1.20.6 yields 0 alarms over 10 control probes.

What a practitioner should do differently

Do not read a reachability-gated relation's alarm as proof: because the reading time is part of the specification, an alarm raised against a crawl-time snapshot cannot be promoted to `violation-proven` on the relation's word alone, and the policy that decides promotion should be reported with the count it produces (Table 16). If an oracle is readable, sound *and* passes the independence admission test of Section VI-E, consult it directly rather than through the relation; a readable,

sound oracle that *fails* that test does not license the substitution, and none of the three products studied passes it. Treat a reported input-level specificity as a corpus statistic rather than a guarantee of scenario-level resolution.

What a stronger claim would require

A population of real systems that *contain* the flaw, which neither the four seeded classes nor the single publicly fixed defect supplies, and relations whose gate is not the absence of evidence. The transferable condition—the *same-observable condition*—is the one that killed our own mechanism: a component's necessity must be tested against a substitute that consumes the *same* observable quantity as the ground truth.

DECLARATIONS

DATA AVAILABILITY

The raw records of the studies reported here are publicly available in the companion repository https://github.com/ubfbuz3/JISA, version-pinned by the tag `access-resubmission-v20`, which archives every record behind every number in this paper, the frozen admission protocol, its errata and the application register (`real_system/prereg/`), and the generation scripts that recompute the tables from those records, including the scoring-rule, ledger-omission and promotion-policy sensitivities (Tables 1, 2 and 16). A previous submission is pinned separately by `access-resubmission-v12`; it carries the records behind the earlier analyses, not the analyses added here and not the paired recrawl experiment of Section V-A, whose record (`results/r15_pair.json`) was produced for this version. Earlier tags (`artifact-v1` and `access-resubmission-v2` through `v17`) are retained as snapshots of prior states and should not be conflated with either. The records are organised under `experiments/`: (1) the scenario/configuration study (`results/raw_matrix.jsonl`, `ablation_direct*.json`, `alt_matrix.jsonl`); (2) the real-system cell study across Gitea 1.22.6, Gogs 0.14.3 and GitLab 17.11.7 (`real_system/results/*.json`), including the three-arm paired recrawl (`r15_pair.json`); (3) the temporal loop and its archived pilot, with the protocol and both amendments by digest (`real_system/results/temporal/`, `c_protocol*.json`); (4) the real vulnerable/fixed pair (`real_system/results/cve_pair.json`); (5) the execution of the published relations on the native engine (`native_engine/results/*.json`); (6) the seeded positive control (`seeded.json`); (7) the equal-budget sweep with its per-cell records (`results/baseline_budget.json`); and (8) the four-class defect study with per-instance and per-channel negative records (`real_system/results/seeded_multiclass.json`). The analysis pipeline (`analysis/*.py`) recomputes every number, table and figure from those records, including the design audit (`ingest_design.py`), the decision-scored coverage table (`ingest_coverage.py`), the truth table (`ingest_truth.py`) and the caliber, ledger and policy sensitivities of Tables 1, 2 and 16 (`ingest_revision.py`); `check_macro_use.py` asserts that every macro cited in the manuscript has exactly one definition. `bash reproduce.sh` regenerates all outputs and the PDF, and `bash verify_artifact.sh` checks the SHA-256 of every raw record and of every byte-reproducible output. The manuscript LaTeX source ships in the same repository. The artifact is archived on Zenodo under the concept DOI https://doi.org/10.5281/zenodo.22859941, which always resolves to the newest version; the git tag above is the version-pinned copy.

COMPETING INTERESTS

The authors declare no competing interests, financial or otherwise.

FUNDING

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

AUTHORS' CONTRIBUTIONS

Qiulong Lin, Xiaoyu Ke, and Nanlei Chen contributed to the work. Qiulong Lin and Nanlei Chen jointly developed the conceptualisation and methodology, designed and implemented the software and analysis pipeline, carried out the investigation and formal analysis, and wrote the manuscript. Xiaoyu Ke guided the experimental work and reviewed the experiments and their analysis.

ETHICS

Not applicable. This study involved no human participants, no personal data, and no animals. All experiments were performed against software systems that we deployed and controlled locally.

DECLARATION OF GENERATIVE AI AND AI-ASSISTED TECHNOLOGIES IN THE WRITING PROCESS

During the preparation of this work the authors used Claude (Anthropic) to assist with literature verification, the development of the analysis scripts, and language editing. Where it contributed to analysis code, every generated or modified script was reviewed, executed, and its output checked by the authors against the raw records. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. All experimental computations and final reported values were independently checked by the authors against the released scripts and raw records. No AI tool participated in the detection loop or in any adjudication.

REFERENCES

[1] OWASP Foundation, "API1:2023 Broken Object Level Authorization," OWASP API Security Top 10, 2023, accessed 18 September 2026. [Online]. Available: https://api-security.owasp.org/editions/2023/en/0xa1-broken-object-level-authorization/

[2] N. B. Chaleshtari, F. Pastore, A. Goknil, and L. Briand, "Metamorphic testing for web system security," *IEEE Transactions on Software Engineering*, pp. 3430–3471, 2023. [Online]. Available: https://doi.org/10.1109/TSE.2023.3256322

[3] P. X. Mai, F. Pastore, A. Goknil, and L. Briand, "Metamorphic security testing for web systems," in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, 2020, pp. 186–197. [Online]. Available: https://doi.org/10.1109/icst46399.2020.00028

[4] R. Croft, M. A. Babar, and M. M. Kholoosi, "Data quality for software vulnerability datasets," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 121–133. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00022

[5] F. Liu, Y. Zhang, E. Li, W. Meng, Y.-Q. Shi, Q. Wang, and M. Yang, "Bacscan: Automatic black-box detection of broken-access-control vulnerabilities in web applications," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, 2025, pp. 1320–1333. [Online]. Available: https://doi.org/10.1145/3719027.3744825

[6] I. P. A. Dharmaadi, M. Alhanahnah, V.-T. Pham, F. Mohsen, and F. Turkmen, "Bacfuzz: Exposing the silence on broken access control vulnerabilities in web applications," *ArXiv*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2507.15984

[7] C. Zuo, Q. Zhao, and Z. Lin, "Authscope: Towards automatic discovery of vulnerable authorizations in online services," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 799–813. [Online]. Available: https://doi.org/10.1145/3133956.3134089

[8] S. E. H. Chehade, F. Hantke, and B. Stock, "403 forbidden? ethically evaluating broken access control in the wild," in *2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 3218–3235. [Online]. Available: https://doi.org/10.1109/SP61157.2025.00252

[9] A. S. Filho, R. J. Rodríguez, and E. Feitosa, "Automated broken object-level authorization attack detection in rest apis through openapi to colored petri nets transformation," *International Journal of Information Security*, vol. 24, no. 2, 2025. [Online]. Available: https://doi.org/10.1007/s10207-024-00970-5

[10] H. Le, L. K. Shar, D. Bianculli, L. Briand, and D. C. Nguyen, "Automated reverse engineering of role-based access control policies of web applications," *Journal of Systems and Software*, vol. 184, p. 111109, 2022. [Online]. Available: https://doi.org/10.1016/j.jss.2021.111109

[11] B. Berger, R. W. Nguempnang, K. Sohr, and R. Koschke, "Static extraction of enforced authorization policies seeauthz," in *2020 IEEE 20th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2020, pp. 187–197. [Online]. Available: https://doi.org/10.1109/SCAM51674.2020.00026

[12] W. Zhang, D. Bali, J. Kerney, A. Panda, and S. Shenker, "Extracting database access-control policies from web applications," in *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI '26)*, 2026, pp. 249–270. [Online]. Available: https://www.usenix.org/conference/osdi26/presentation/zhang-wen

[13] Y.-Q. Shi, F. Liu, G. Yang, Y. Zhang, Y. Cao, E. Li, and S. Chen, "Facilitating access control vulnerability detection in modern java web applications with accurate permission check identification," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 10 683–10 698, 2025. [Online]. Available: https://doi.org/10.1109/TIFS.2025.3614424

[14] Y. Huang, C. Shi, J. Lu, H. Li, H. Meng, and L. Li, "Detecting broken object-level authorization vulnerabilities in database-backed applications," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 2934–2948. [Online]. Available: https://doi.org/10.1145/3658644.3690227

[15] F. Liu, Y.-Q. Shi, Y. Zhang, G. Yang, E. Li, and M. Yang, "Mocguard: Automatically detecting missing-owner-check vulnerabilities in java web applications," in *2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 903–919. [Online]. Available: https://doi.org/10.1109/SP61157.2025.00010

[16] R. Feng, M. Zhang, A. Xia, J. Cheng, Y. Li, Y. Zhang, and Y. Liu, "Jauthguard: Automatic detection for broken access control in java web apis," in *2025 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2025, pp. 3102–3107. [Online]. Available: https://doi.org/10.1109/SMC58881.2025.11343625

[17] M. Schlaubitz, O. Veyisoglu, and M. Rennhard, "A2ct: Automated detection of function and object-level access control vulnerabilities in web applications," in *International Conference on Information Systems Security and Privacy*, 2025, pp. 425–436. [Online]. Available: https://doi.org/10.5220/0013092700003899

[18] P. Li, H. Chong, Y. Cao, and J. Yang, "Detecting privilege escalation in polyglot microservices via agentic program analysis," in *2026 IEEE Symposium on Security and Privacy (SP)*, 2026, pp. 1747–1765. [Online]. Available: https://doi.org/10.1109/SP63933.2026.00121

[19] V. Atlidakis, P. Godefroid, and M. Polishchuk, "Restler: Stateful rest api fuzzing," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 2019, pp. 748–758. [Online]. Available: https://doi.org/10.1109/ICSE.2019.00083

[20] A. Arcuri, M. Zhang, and J. P. Galeotti, "Advanced white-box heuristics for search-based fuzzing of rest apis," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 6, pp. 1–36, 2024. [Online]. Available: https://doi.org/10.1145/3652157

[21] S. Seran, M. Zhang, O. Duman, and A. Arcuri, "Handling web service interactions in fuzzing with search-based mock-generation," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 2, pp. 1–45, 2026. [Online]. Available: https://doi.org/10.1145/3731558

[22] Ömür Şahin, M. Zhang, and A. Arcuri, "Enhancing rest api fuzzing with access policy violation checks and injection attacks," *Journal of Systems and Software*, vol. 243, p. 113060, 2027. [Online]. Available: https://doi.org/10.1016/j.jss.2026.113060

[23] M. A. Uddin, S. Weerasinghe, C. Wojtak, T. Cerný, D. Silva-Junior, M. E. S. Ribeiro, and A. S. Abdelfattah, "Automated generation of microservice authorization tests using large language models," in *2026 International Conference on Service-Oriented System Engineering (SOSE)*, 2026, pp. 41–50. [Online]. Available: https://doi.org/10.1109/SOSE71128.2026.00014

[24] J. Sun, R. Tian, S. Zhao, X. Zhou, H. Wang, and Y. Zhang, "Llm-assisted idor detection in hospital mini-programs: Risks to pii and phi," in *2025 IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, 2025, pp. 2078–2086. [Online]. Available: https://doi.org/10.1109/Trustcom66490.2025.00242

[25] E. Wang, W. Xie, S. Li, R. Liu, Y. Zhou, Z. Wang, and B. Wang, "Large language model-powered protected interface evasion: Automated discovery of broken access control vulnerabilities in internet of things devices," *Sensors (Basel, Switzerland)*, vol. 25, no. 9, p. 2913, 2025. [Online]. Available: https://doi.org/10.3390/s25092913

[26] A. Wu, Z. Feng, R. Feng, Z. Xing, and Y. Liu, "Rethinking broken object level authorization attacks under zero trust principle," *ACM Transactions on Software Engineering and Methodology*, 2026. [Online]. Available: https://doi.org/10.1145/3833417

[27] H. Wang, Z. Wang, and P. Liu, "A hybrid llm workflow can help identify user privilege related variables in programs of any size," *ArXiv*, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2403.15723

[28] C. Wen, J. Cao, J. Su, Z. Xu, S. Qin, M. He, and C. Tian, "Enchanting program specification synthesis by large language models using static analysis and program verification," in *International Conference on Computer Aided Verification*, 2024, pp. 302–328. [Online]. Available: https://doi.org/10.1007/978-3-031-65630-9_16

[29] R. Wang, H. Zhuang, H. Cao, and S. Chen, "Measuring the reasoning boundaries of large language models for implicit security invariants in code: A controlled empirical study," in *2026 IEEE 50th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2026, pp. 3146–3157. [Online]. Available: https://doi.org/10.1109/COMPSAC69091.2026.00467

[30] T. Le-Cong, B. Le, and T. Murray, "Can llms reason about program semantics? a comprehensive evaluation of llms on formal specification inference," in *Annual Meeting of the Association for Computational Linguistics*, 2025, pp. 21 991–22 014. [Online]. Available: https://aclanthology.org/2025.acl-long.1068/

[31] J. Liu, G. Lin, H. Mei, F. Yang, and Y. Tai, "Enhancing vulnerability detection efficiency: An exploration of light-weight llms with hybrid code features," *Journal of Information Security and Applications*, vol. 88, p. 103925, 2024. [Online]. Available: https://doi.org/10.1016/j.jisa.2024.103925

[32] J. Senanayake, H. Kalutarage, A. Petrovski, L. Piras, and M. O. Al-Kadri, "Defendroid: Real-time android code vulnerability detection via blockchain federated neural network with xai," *Journal of Information Security and Applications*, vol. 82, p. 103741, 2024. [Online]. Available: https://doi.org/10.1016/j.jisa.2024.103741

QIULONG LIN Qiulong Lin received the M.S. degree from Sogang University, Seoul, Republic of Korea. He is with Yango University, Fuzhou, China, and is also affiliated with Sogang University, Seoul, Republic of Korea. His research interests include Web/API security, vulnerability discovery, and metamorphic testing.

XIAOYU KE Xiaoyu Ke is with Yango University, Fuzhou, China.

NANLEI CHEN Nanlei Chen is currently pursuing the M.S. degree at Sogang University, Seoul, Republic of Korea. He is with Yango University, Fuzhou, China. His research interests include Web/API security and authorization testing. He is the corresponding author of this work.